

使用Python自动化管理和分析空间数据



Esri中国 刘利平

自动化处理概述

自动化处理应用场景

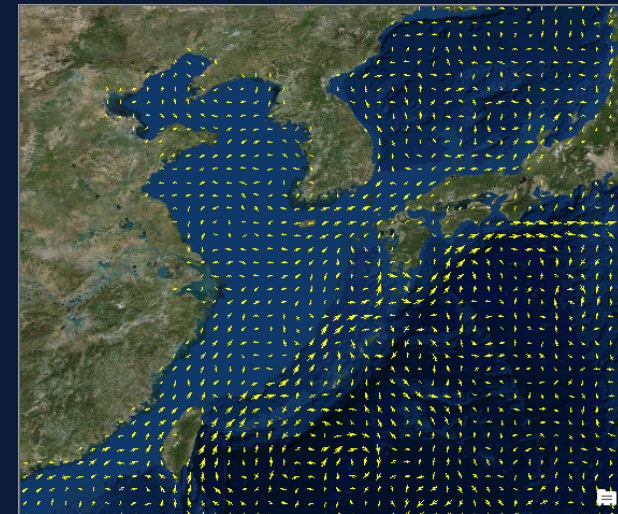
使用场景

- 特点

- 重复性
- 流程化
- 部分人工辅助

- 适用场景

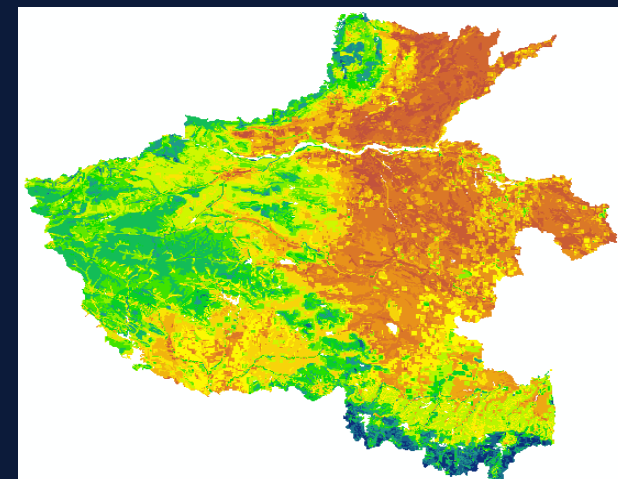
- 数据自动化处理（监测、模型）
- 自动化制图
- 模型评估分析
-



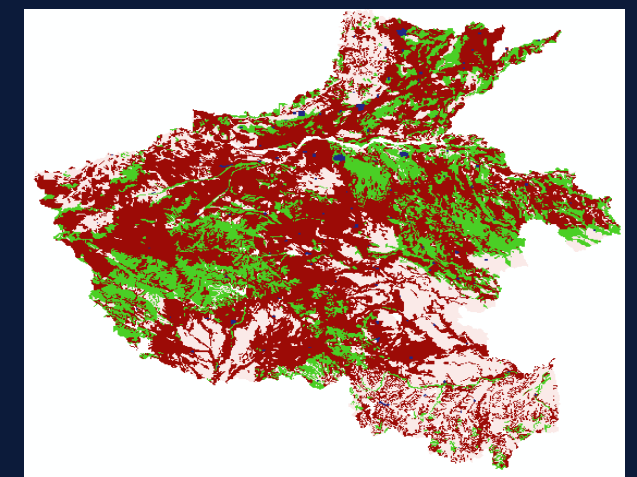
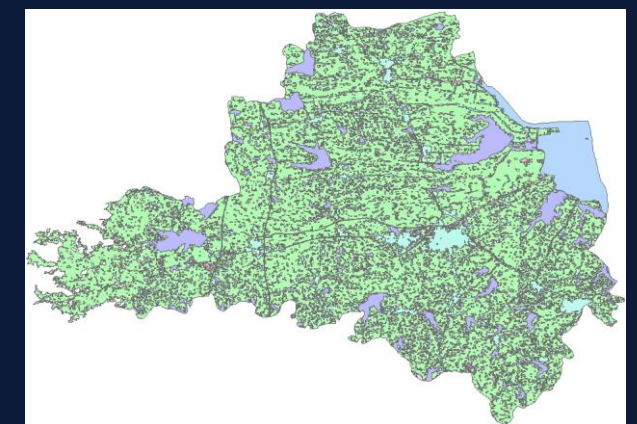
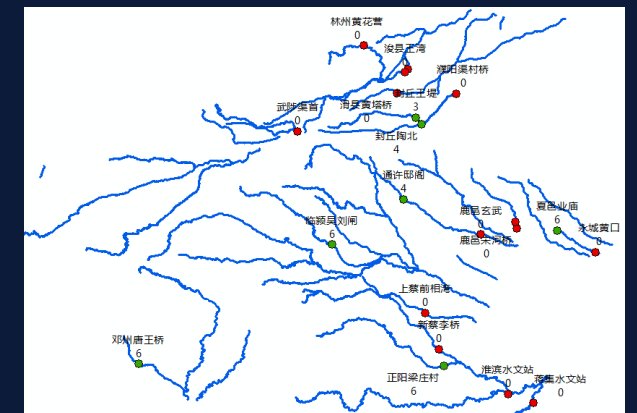
洋流预报数据



水质自动化制图



Geodatabase空间数据模型



自动化处理对象

非空间数据

- 文本类型数据

- GRD

- Txt

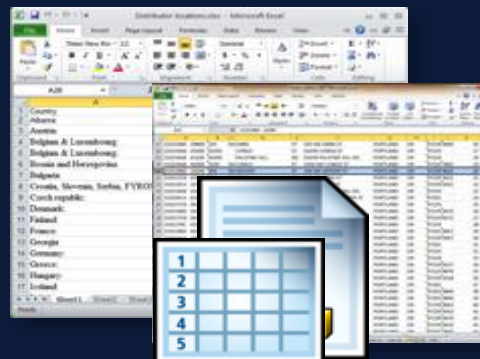
- Excel

- 结构化数据

- Oracle\SQL Server\DB2\PostgreSQL

- MySQL

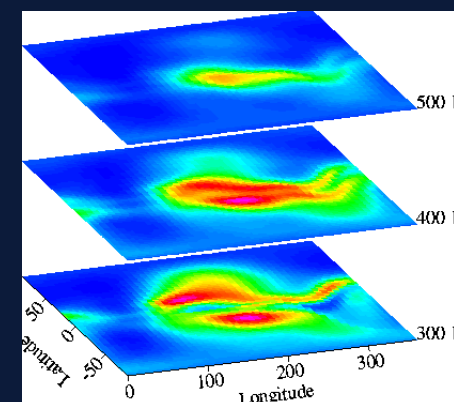
- 科学多维数据



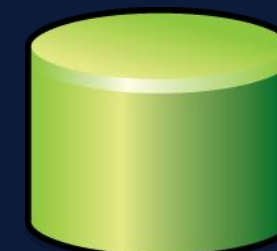
电子表格数据

DSAA	201	91		
	1	201		
	1	91		
	98729.27	103418.3		
	100989.9	100995.1	101033.5	101097.5
	101001.9	100980.1	100976.7	100984.3
	100978.1	100961.7	101035.3	101205.7
	101449.1	101621.9	101518.1	101383.1
	101315.7	101290.5	101281.5	101273.9
	101272.9	101276.3	101278.7	101286.3
	101302.5	101302.1	101301.7	101304.7
	101317.7	101320.5	101329.5	101331.7
	101359.1	101388.1	101350.7	101431.9
	101341.7	101353.5	101363.9	101364.3
	101369.7	101387.3	101389.5	101385.1
	101373.3	101379.5	101383.5	101381.3
	101366.3	101359.9	101358.3	101361.3
	101375.5	101384.9	101391.3	101395.5
	101400.5	101397.1	101389.9	101380.3
	101361.5	101352.7	101343.9	101335.9

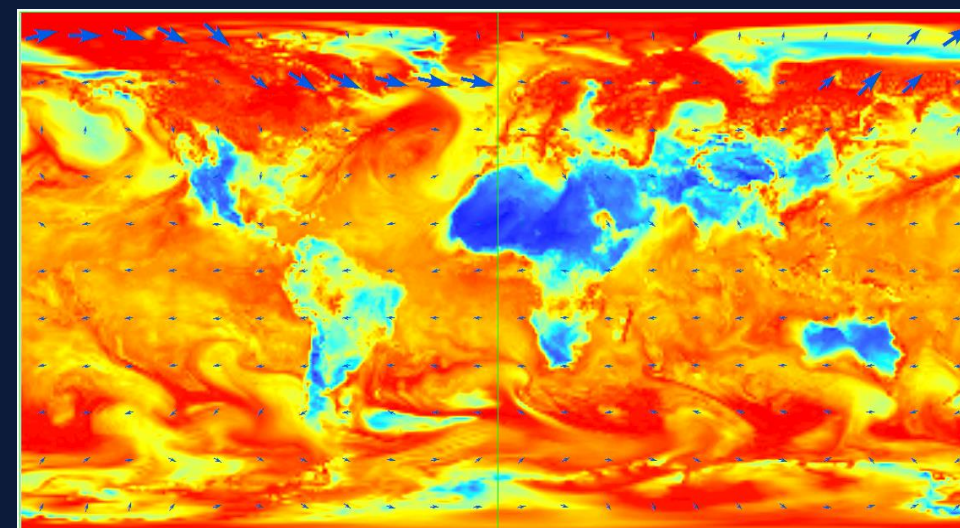
GRD数据



多维数据

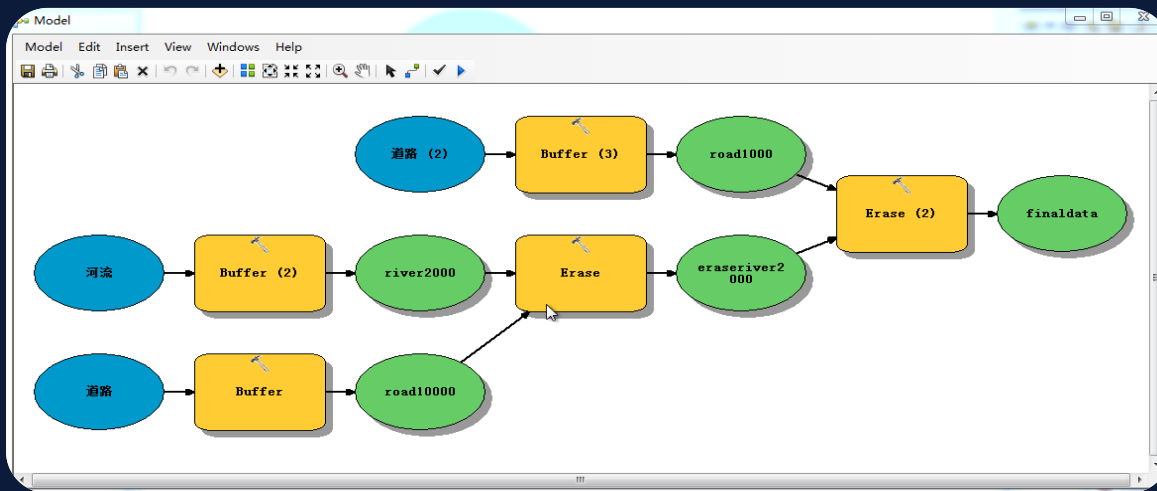


数据库中
非空间数据



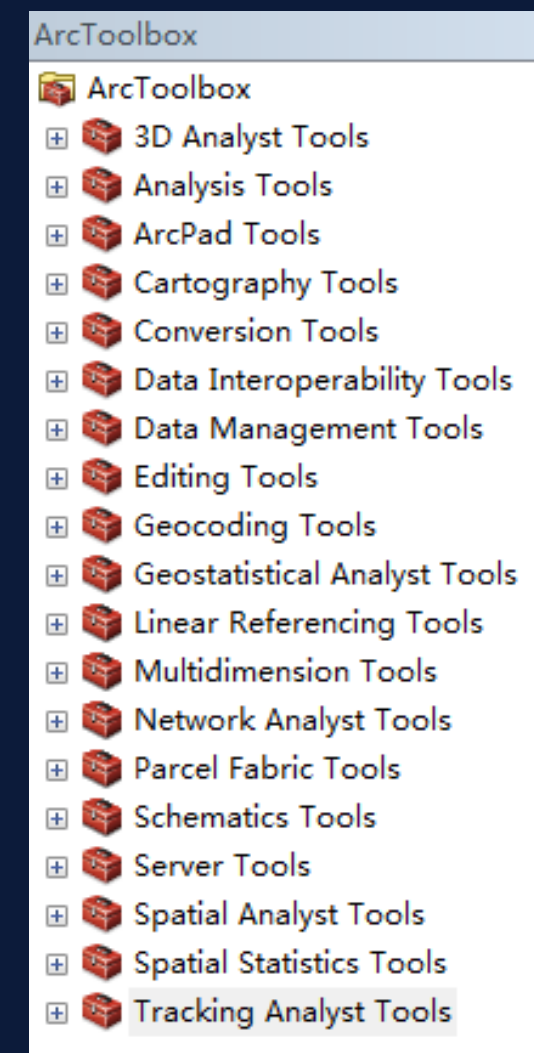
自动化处理方法

- Geoprocessing框架
 - ArcToolbox (ArcGIS提供的地理分析处理工具集)
 - Model Builder (使用一系列GP工具构建工作流的可视化编程语言)
 - Python-Arcpy (使用Arcpy操作地理处理工具和自带函数方法 , 实现更强大的地理处理工具和程序)



```
# Local variables:
lakes = "lakes"
lakes_grid = "lakes_grid"
lakes_Clip = "C:\\Users\\Administrator\\Documents\\

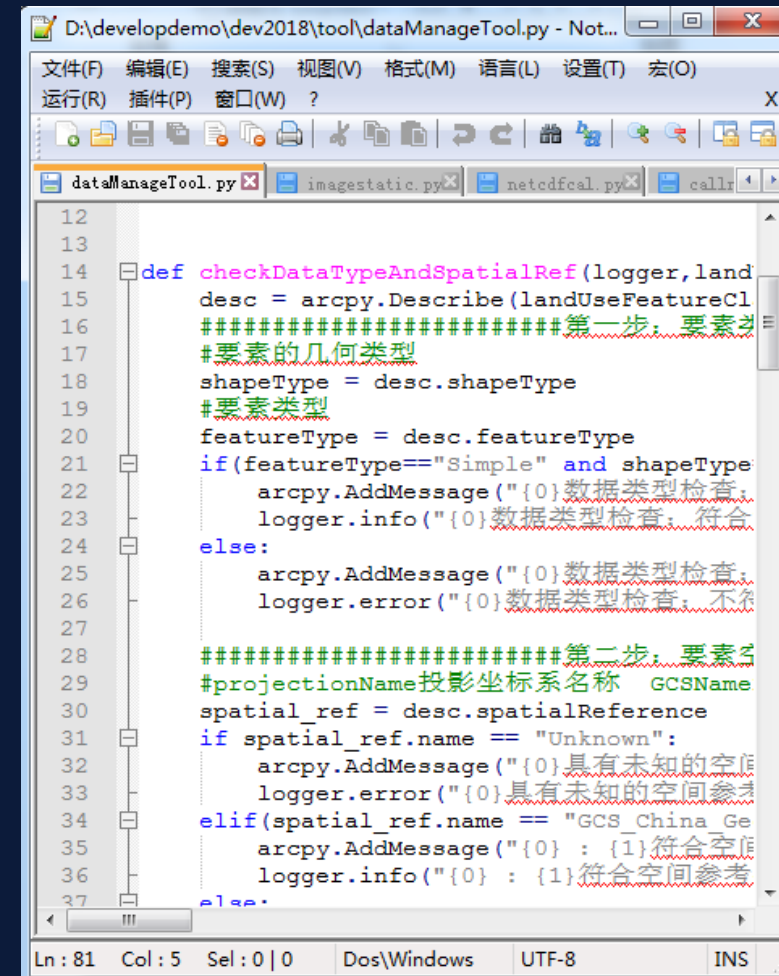
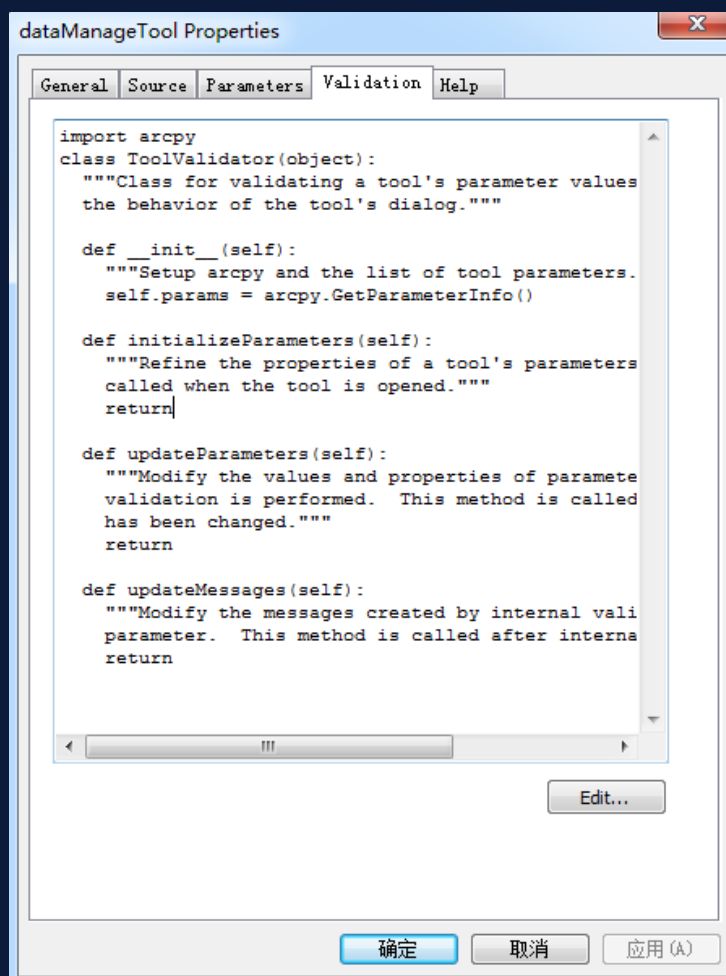
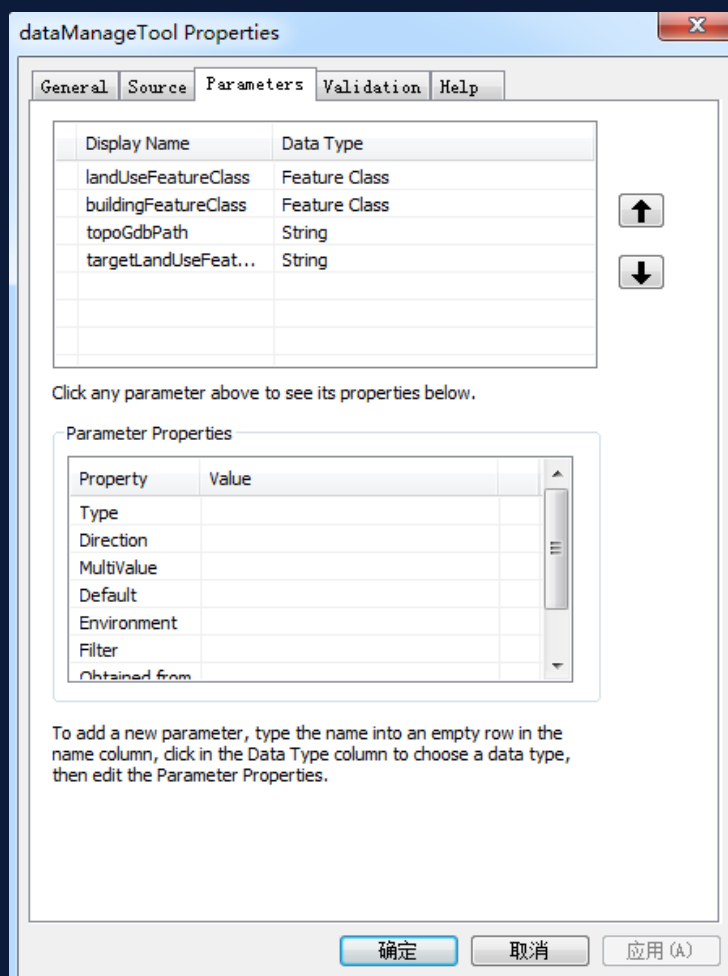
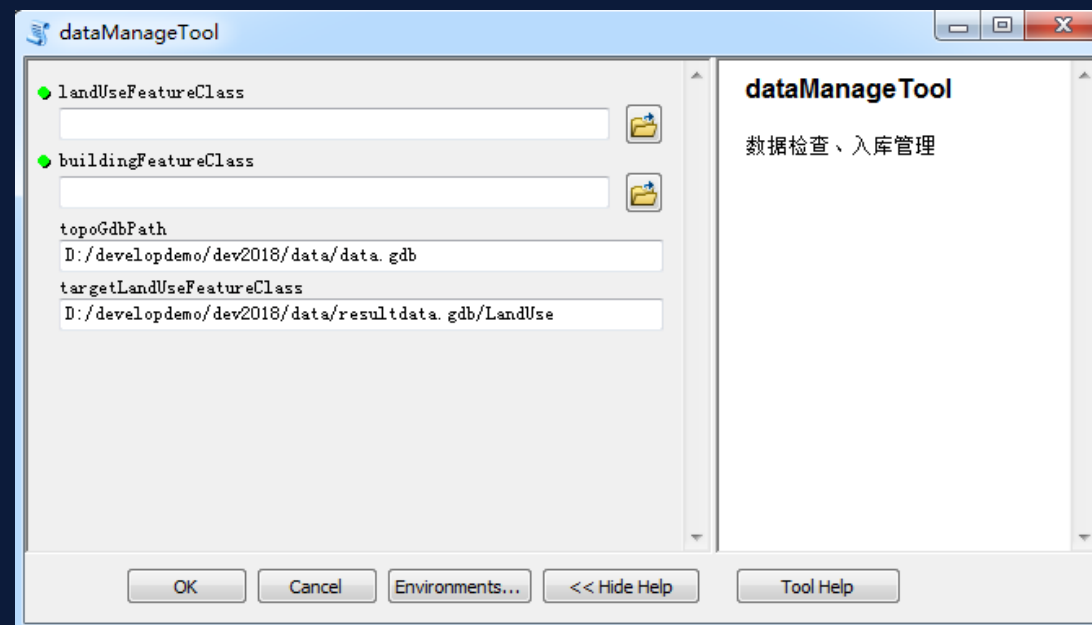
# Process: 裁剪
arcpy.Clip_analysis(lakes, lakes_grid, lakes_Clip,
```



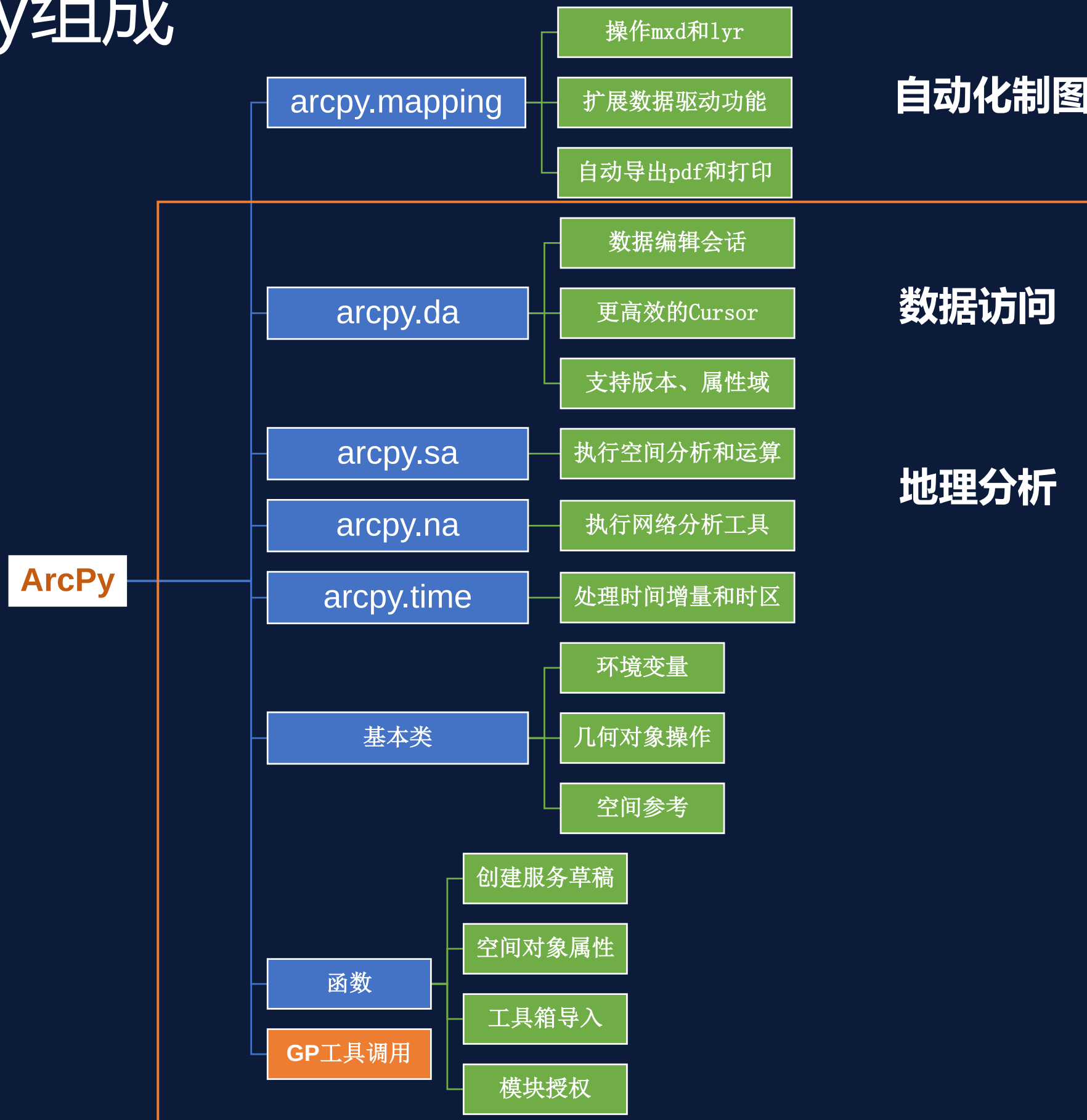
自动化处理方法

- 自定义GP工具

- 向导创建参数
- Toolbox中python脚本验证
- Python处理源码



Arcpy组成



集成第三方Python库

集成科学计算NumPy——扩展分析和处理地理数据

- 关于NumPy

- 强大的N维数组对象，存储和处理大型矩阵（灵活索引、合并拆分）
- 复杂分析计算的能力（算术运算、维度变换、线性函数）
- 与netCDF，Pandas，Scikit-learn等科学计算库交互运算

- arcpy.da支持numpy arrays与table，feature class的相互转化

- *TableToNumPyArray, NumPyArrayToTable*
- *FeatureClassToNumPyArray, NumPyArrayToFeatureClass*

- 同时支持raster与numpy arrays相互转化，属于arcpy命名空间

- *RasterToNumPyArray, NumPyArrayToRaster*

- ArcMap，Server，Engine10.4，Pro1.1集成Pandas包

```
C:\Python27\ArcGIS10.6\Scripts>pip list
DEPRECATION: The default format will switch to columns in your pip.conf
cycler (0.10.0)
functools32 (3.2.3.post2)
future (0.15.2)
matplotlib (1.5.2)
mpmath (0.19)
nose (1.3.7)
numpy (1.9.3)
pandas (0.18.1)
pip (9.0.1)
pyparsing (2.1.4)
python-dateutil (2.5.3)
pytz (2016.6.1)
requests (2.11.1)
setuptools (17.0)
setuptools (28.8.0)
six (1.10.0)
sympy (1.0)
wheel (0.29.0)
xlrd (1.0.0)
xlwt (1.1.2)
```

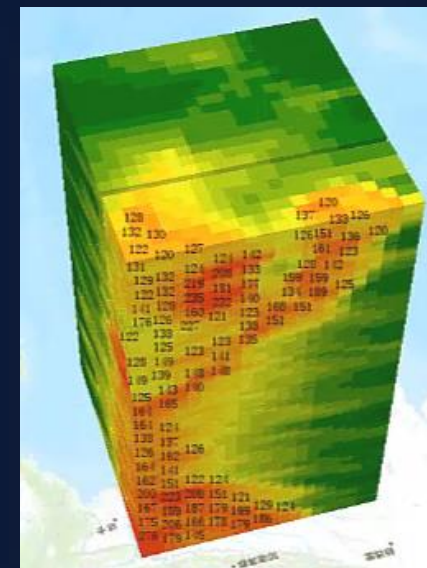
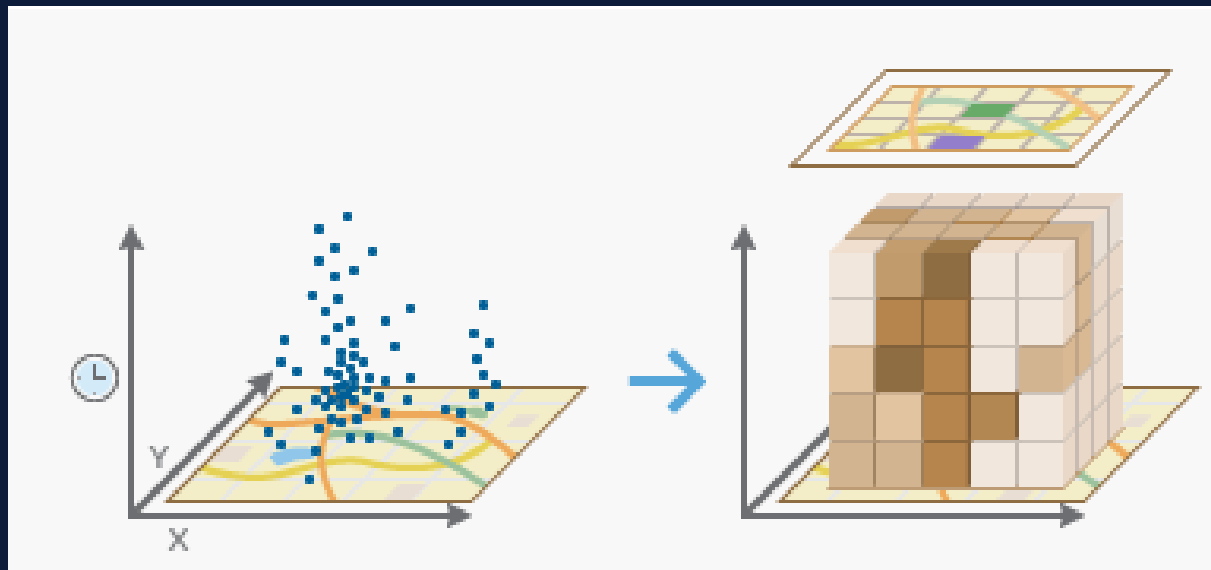
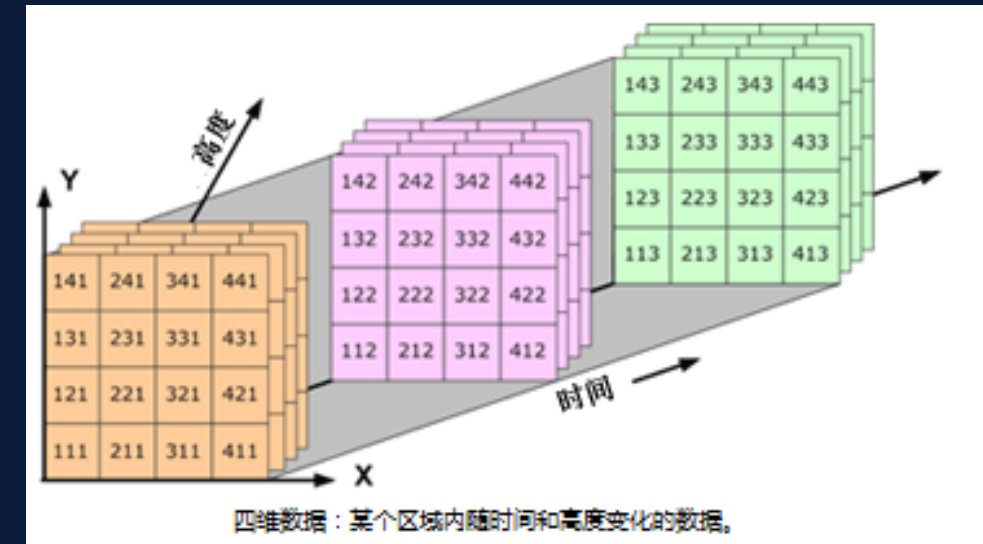


```
>>> a[3:,[0,2,5]]
array([[30,32,35],
       [40,42,45],
       [50,52,55]])
```

集成其他第三方Python库

安装多维数据netCDF包

- 关于netCDF
 - 多维科学数据，常见于海洋、气象、环境等行业应用
 - 二进制、自描述的文件格式
 - ArcGIS10.3支持多维数据镶嵌数据集
 - 时空立方体/新兴热点分析

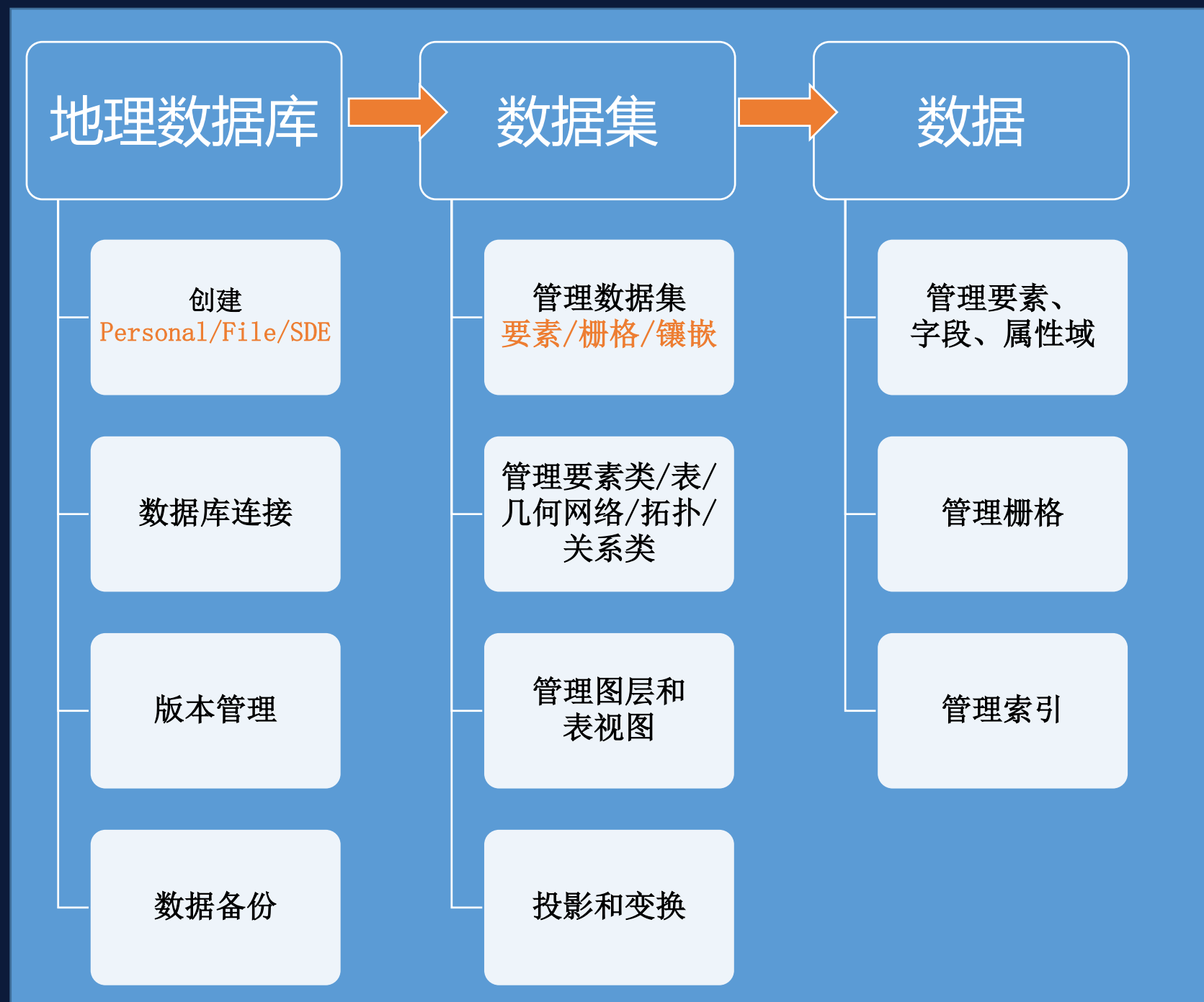


数据处理与管理

数据管理

数据管理应用

- 附件工具集
- 存档工具集
- 数据比较工具集
- 分布式地理数据库 (Geodatabase) 工具集
- 属性域工具集
- 要素类工具集
- 要素工具集
- 字段工具集
- 文件地理数据库 (File Geodatabase) 工具集
- “地理数据库管理”工具集
- 常规工具集
- 制图综合工具集
- 几何网络 (Geometric Network) 工具集
- 图表工具集
- 索引工具集
- 连接工具集
- “LAS 数据集”工具集
- 图层和表视图工具集
- 打包工具集
- 照片工具集
- 投影和变换工具集
- 栅格工具集
- 关系类工具集
- 子类型工具集
- 表工具集
- 切片缓存工具集
- 拓扑工具集
- 版本工具集
- Workspace toolset



管理工具使用注意事项

产品版本授权

产品版本要求： `arcpy.CheckProduct("ArcInfo")`

Parameter	Explanation
product	Product code for the product being checked. • arcview —ArcGIS Desktop Basic product code • arceditor —ArcGIS Desktop Standard product code • arcinfo —ArcGIS Desktop Advanced product code • engine —Engine Runtime product code • enginegeodb —Engine Geodatabase Update product code • arcserver — Server product code

Data Type	Explanation
String	There are five possible return values for CheckProduct: • AlreadyInitialized — License has already been set in the script. • Available — The requested license is available to be set. • Unavailable — The requested license is unavailable to be set. • NotLicensed — The requested license is not valid. • Failed — A system failure occurred during the request.

```
#验证产品版本权限
if arcpy.CheckProduct("ArcInfo") == "Available":
    arcpy.AddMessage('ArcGIS for Desktop Advanced license is available')
elif arcpy.CheckProduct("ArcInfo") == "AlreadyInitialized":
    arcpy.AddMessage('ArcGIS for Desktop Advanced license is AlreadyInitialized')
else:
    arcpy.AddMessage('ArcGIS for Desktop Advanced license not available')
```

要素 工具集			
添加几何属性	✓	✓	✓
添加 XY 坐标	✓	✓	✓
调整 3D Z 值	✓	✓	✓
原点夹角距离定义线	✓	✓	✓
检查几何	✓	✓	✓
复制要素	✓	✓	✓
删除要素	✓	✓	✓
切分			✓
要素包络矩形转面			✓
要素折点转点			✓
要素转线			✓
要素转点			✓
要素转面			✓
测地线增密		✓	✓
最小边界几何	✓	✓	✓
多部件至单部件	✓	✓	✓
点集转线	✓	✓	✓
面转线			✓
修复几何	✓	✓	✓
在点处分割线			✓
在折点处分割线			✓
表转椭圆	✓	✓	✓
取消线分割			✓

arcpy.da

数据访问模块

- 支持要素类、表的记录的增删改查

SearchCursor	数据读取
UpdateCursor	数据更新和删除
InsertCursor	数据插入

- arcpy.da Cursors支持with声明
 - 提供上下文管理
 - 代码简洁清晰

```
• with arcpy.da.SearchCursor(table, field) as cursor:  
•     for row in cursor:  
•         print row[0]
```

arcpy.da

数据访问模块

Cursor

```
arcpy.da.InsertCursor(in_table, field_names)
```

```
arcpy.da.SearchCursor(in_table, field_names, {where_clause}, {spatial_reference}, {explode_to_points}, {sql_clause})
```

```
arcpy.da.UpdateCursor(in_table, field_names, {where_clause}, {spatial_reference}, {explode_to_points}, {sql_clause})
```

```
with arcpy.da.InsertCursor(tracelinefullname, ['shape@', 'EventID', 'Time']) as lrows:
    for i in range(8):
        sublist = newtracepointlist[i*ptmax:(i+1)*ptmax]
        #获取轨迹线id
        eventid = sublist[0][0]
        #获取轨迹线开始时间
        sdate = sublist[0][1]
        #构造线对象
        linegeometry = arcpy.Polyline(arcpy.Array([arcpy.Point(coords[2], coords[3]) for coords in sublist]))
        lrows.insertRow([linegeometry, eventid, sdate])
```

• 参数说明

Field_names

- 可以使用*访问所有字段
- 字段列表限制在实际需要字段（性能更高，控制字段顺序）
- 几何属性可以通过SHAPE@进行访问
- 使用SHAPE@访问整个几何需要更高代价
- 数据库特殊Sql，DISTINCT、ORDER BY 和GROUP BY

```
for row in arcpy.da.SearchCursor(
    fc, fields, sql_clause=(None, 'ORDER BY WELL_ID, WELL_TYPE')):
```

读取几何

- 'OID@'
- 'SHAPE@XY'
- 'SHAPE@TRUECENTROID'
- 'SHAPE@X'
- 'SHAPE@Y'
- 'SHAPE@Z'
- 'SHAPE@M'
- 'SHAPE@JSON'
- 'SHAPE@WKB'
- 'SHAPE@WKT'
- 'SHAPE@'
- 'SHAPE@AREA'
- 'SHAPE@LENGTH'

arcpy.da Cursors VS arcpy Cursors

数据访问模块

- arcpy.da Cursors

- 使用list和tuple
- 通过索引访问row值
- 效率更高

```
with arcpy.da.UpdateCursor("c:/base/data.gdb/roads",  
                           ["roadtype", "distance"]) as cursor:  
    for row in cursor:  
        # Update the values in the distance field by multiplying  
        # the roadtype by 100. Road type is either 1, 2, 3 or 4.  
        #  
        row[1] = row[0] * 100  
        cursor.updateRow(row)
```

```
• fields = ['field1', 'field2']  
• cursor = arcpy.da.InsertCursor(table, fields)  
• cursor.insertRow([1, 10])
```

- arcpy Cursors

- 用于10.1版本之前的脚本
- 使用Row对象
- 通过setValue和getValue赋值

```
• cursor = arcpy.InsertCursor(table)  
• row = cursor.newRow()  
• row.setValue("field1", 1)  
• row.setValue("field2", 10)  
• cursor.insertRow(row)
```

写入几何

写入方法

- Geometry objects
- Esri JSON
- List of coordinates

✓ Multipoint

✓ Point

✓ Polygon

✓ Polyline

```
• cursor = arcpy.da.InsertCursor(fc, 'SHAPE@')
• line = arcpy.Polyline(arcpy.Array([arcpy.Point(-7216000.0, 5944500.0),
•                                     arcpy.Point(-7225700.0, 5934500.0)]),
•                                     arcpy.SpatialReference(3857))
• cursor.insertRow([line])
```

```
• cursor = arcpy.da.InsertCursor(fc, 'SHAPE@JSON')
• json_line = {"paths": [[[-7216000.0, 5944500.0], [-7225700.0, 5934500.0]]],
•              "spatialReference": {"wkid": 102100, "latestWkid": 3857}}
• cursor.insertRow([json.loads(json_line)])
```

```
• cursor = arcpy.da.InsertCursor(fc, 'SHAPE@')
• coordinate_list = [(-7216000.0, 5944500.0), (-7225700.0, 5934500.0)]
• cursor.insertRow([coordinate_list])
```

arcpy.da.Editor

| 使用场景

- 事务处理
 - 批量事务提交
 - 多用户操作
 - 撤销重做处理
- 必须使用
 - 企业级地理数据库中版本化数据集
 - 参与拓扑的要素类
 - 参与几何网络的要素类，如汇点、边界线

arcpy.da.Editor

| With支持

```
• with arcpy.da.Editor(workspace) as edit:  
    # your edits
```

← 打开编辑会话开始编辑操作



异常：操作被取消，编辑会话关闭，不保存操作

无异常：停止操作，保存并关闭编辑会话

Editor methods

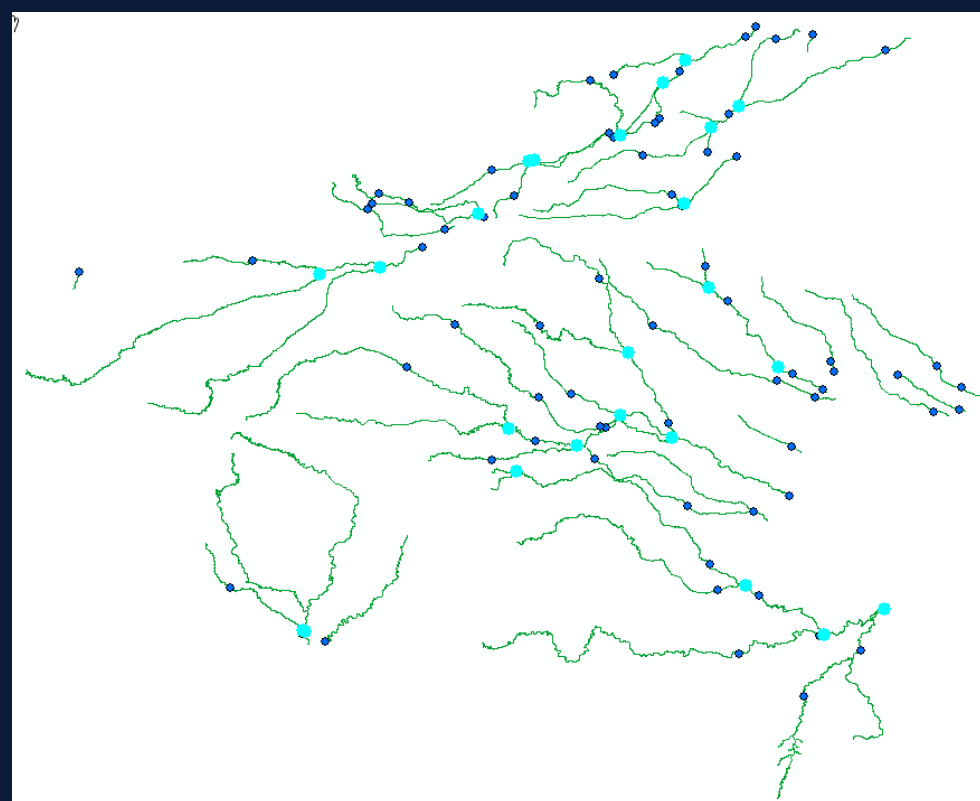
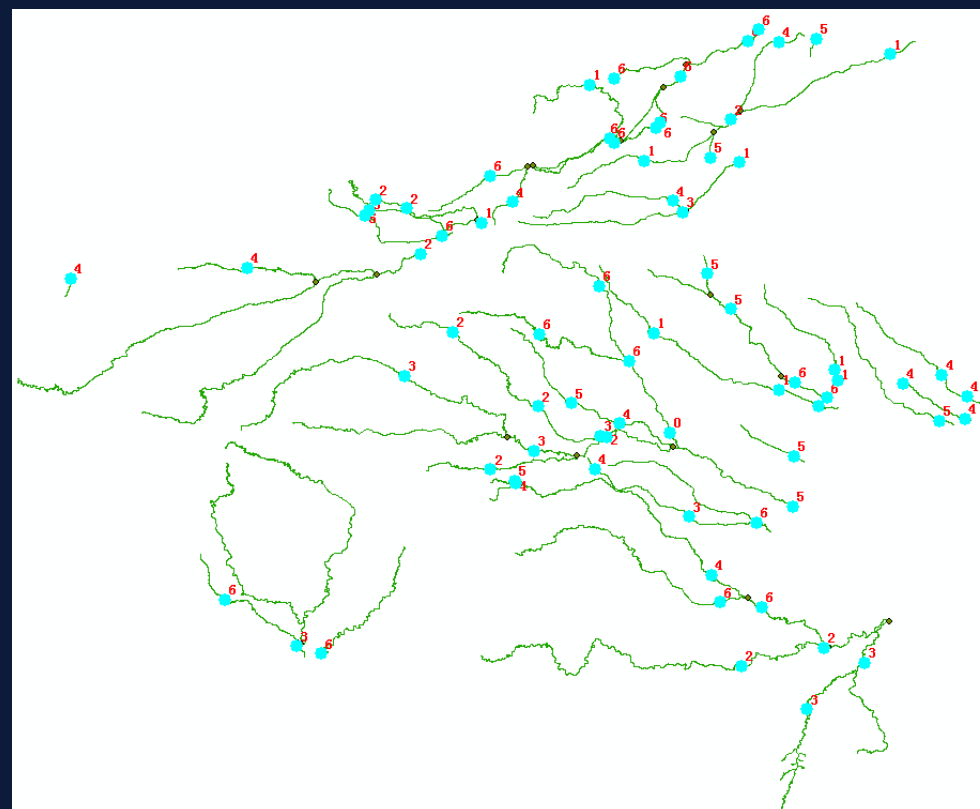
startEditing ({with_undo}, {multiuser_mode})	Starts an edit session.
stopEditing(save_changes)	Stops an edit session.
startOperation()	Starts an edit operation.
stopOperation()	Stops an edit operation.
abortOperation()	Aborts an edit operation.
undoOperation()	Undo an edit operation (roll back modifications).
redoOperation()	Redoes an edit operation.

版本数据更新和NumPy应用

版本数据编辑

OBJECTID *	sitename	district	value	ison	val	myid	parentid	x	y
1	浚县崇湾	鹤壁	0	1	6	81	<Null>	114.527496	35.60944
2	浚县王湾	鹤壁	0	1	6	82	<Null>	114.499672	35.579292
3	鹿邑宋河桥	鹿邑	0	1	1	83	<Null>	115.610687	34.040901
4	鹿邑玄武	鹿邑	0	1	1	84	<Null>	115.252922	33.982811
5	新蔡李桥	新蔡	0	1	4	85	<Null>	114.837311	32.849491
6	上蔡前相湾	周口	0	1	3	86	<Null>	114.703941	33.206028
7	封丘陶北	长垣	4	1	3	87	<Null>	114.66423	35.06559
8	封丘王堤	长垣	3	1	4	88	<Null>	114.606003	35.133457
9	邓州唐王桥	邓州	6	1	6	89	<Null>	111.866707	32.701664
10	永城黄口	永城	0	1	4	90	<Null>	116.386375	33.80584
11	蒋集水文站	信阳	0	1	3	91	<Null>	115.77343	32.315762
12	正阳梁庄村	新蔡	6	1	6	92	<Null>	114.888	32.68544
13	淮滨水文站	信阳	0	1	2	93	<Null>	115.521935	32.406837
14	睢阳包公庙	商丘	0	1	1	94	<Null>	115.592011	34.101627
15	濮阳梁村桥	濮阳	0	1	1	95	<Null>	115.005692	35.36858
16	通许郎阁	周口	4	1	1	96	<Null>	114.484512	34.327332
17	临颍吴刘闸	漯河	6	1	2	97	<Null>	113.779739	33.879658
18	夏邑业庙	永城	6	1	4	98	<Null>	116.005783	34.021088
19	滑县黄塔桥	滑县	0	1	1	99	<Null>	114.426338	35.377411
20	林州黄花香	安阳	0	2	1	100	<Null>	114.095863	35.843586
21	大名龙王庙	<Null>	<Null>	<Null>	6	0	<Null>	115.125221	36.177422
22	鹿邑付桥	<Null>	<Null>	<Null>	3	2	<Null>	115.489563	33.880356
23	新蔡班台	<Null>	<Null>	<Null>	6	3	<Null>	115.14621	32.653244
24	鹿邑东孙营	<Null>	<Null>	<Null>	6	6	<Null>	115.542236	33.930958
25	永城马桥	<Null>	<Null>	<Null>	5	7	<Null>	116.227058	33.791443
26	西华大王庄	<Null>	<Null>	<Null>	0	8	<Null>	114.583565	33.718414

OBJECTID *	sitename	district	value	ison	val	myid	parentid	x	y
97	<Null>	<Null>	<Null>	<Null>	0	106	55	<Null>	<Null>
96	<Null>	<Null>	<Null>	<Null>	0	104	85, 92	115.059753	32.713634
95	<Null>	<Null>	<Null>	<Null>	0	102	17, 100	114.283607	35.502037
94	<Null>	<Null>	<Null>	<Null>	0	101	49	112.420639	34.64201
93	<Null>	<Null>	<Null>	<Null>	0	80	78, 91	115.921799	32.569984
92	<Null>	<Null>	<Null>	<Null>	0	78	3, 93	115.54763	32.40937
91	<Null>	<Null>	<Null>	<Null>	0	77	30	115.265289	34.06707
90	<Null>	<Null>	<Null>	<Null>	0	74	51	115.017342	35.685368
89	<Null>	<Null>	<Null>	<Null>	0	72	44, 99	114.849167	35.552555
88	<Null>	<Null>	<Null>	<Null>	0	71	22	114.833832	34.560287
87	<Null>	<Null>	<Null>	<Null>	0	70	14, 15	114.685425	35.967224
86	<Null>	<Null>	<Null>	<Null>	0	69	87, 88	114.68277	35.079189
85	<Null>	<Null>	<Null>	<Null>	0	67	40, 64	114.605591	33.634033
84	<Null>	<Null>	<Null>	<Null>	0	66	81, 102	114.546577	35.827297
83	<Null>	<Null>	<Null>	<Null>	0	65	23, 33	114.331795	34.159882
82	<Null>	<Null>	<Null>	<Null>	0	64	38, 41	114.285286	33.772247
81	<Null>	<Null>	<Null>	<Null>	0	63	25, 28	114.015938	33.57962
80	<Null>	<Null>	<Null>	<Null>	0	62	61	113.751152	35.350624
79	<Null>	<Null>	<Null>	<Null>	0	61	16, 18	113.718384	35.343185
78	<Null>	<Null>	<Null>	<Null>	0	60	13, 27	113.641785	33.422657
77	<Null>	<Null>	<Null>	<Null>	0	59	29	113.592438	33.689171
76	<Null>	<Null>	<Null>	<Null>	0	58	47, 57	113.408653	35.01804
75	新野刁河堂	<Null>	<Null>	<Null>	0	55	89	112.314255	32.439262
74	伊洛汇合处	<Null>	<Null>	<Null>	0	9	101	112.792969	34.682293



版本数据处理和NumPy应用

NumPy与FeatureClass转换

```
#监测站点
pointfeature = arcpy.GetParameterAsText(0)
pointfields = ['MYID', 'VAL', 'X', 'Y', 'PARENTID']
pointfeature = "D:/EnvironmentApplication/Connection to localhost.sde/mypostgresdb.sde.XYpoints_all"
#FeatureClass转换成Numpy
pointwhereclause = ""
pointArr = arcpy.da.FeatureClassToNumPyArray(pointfeature, pointfields, pointwhereclause)
pointList = pointArr.tolist()
pointdatatype = numpy.dtype({'names': pointfields, 'formats': [numpy.int32, numpy.float64, numpy.float64, numpy.float64, 'S32']})
pointArr = numpy.array(pointList, dtype=pointdatatype)

#构建新增汇点,根据parentid是否为空,得到所有的需要计算点
interzeroarr = zeros((pointArr.size), dtype=pointdatatype)
intercount = 0
for pointitem in pointArr:
    if str(pointitem[4]).strip() != 'None' and str(pointitem[4]).strip() != '':
        interzeroarr[intercount] = pointitem
        intercount = intercount + 1
interpointarr = interzeroarr[0:intercount]
```

OBJECTID *	sitename	district	value	ison	val	myid	parentid	x	y
97	<Null>	<Null>	<Null>	<Null>	0	106	55	<Null>	<Null>
96	<Null>	<Null>	<Null>	<Null>	0	104	85, 92	115.059753	32.713634
95	<Null>	<Null>	<Null>	<Null>	0	102	17, 100	114.283607	35.502037
94	<Null>	<Null>	<Null>	<Null>	0	101	49	112.420639	34.64201
93	<Null>	<Null>	<Null>	<Null>	0	80	78, 91	115.921799	32.569984
92	<Null>	<Null>	<Null>	<Null>	0	78	3, 93	115.54763	32.40937
91	<Null>	<Null>	<Null>	<Null>	0	77	30	115.265289	34.06707
90	<Null>	<Null>	<Null>	<Null>	0	74	51	115.017342	35.685368
89	<Null>	<Null>	<Null>	<Null>	0	72	44, 99	114.849167	35.552555
88	<Null>	<Null>	<Null>	<Null>	0	71	22	114.833832	34.560287
87	<Null>	<Null>	<Null>	<Null>	0	70	14, 15	114.685425	35.967224
86	<Null>	<Null>	<Null>	<Null>	0	69	87, 88	114.68277	35.079189
85	<Null>	<Null>	<Null>	<Null>	0	67	40, 64	114.605591	33.634033
84	<Null>	<Null>	<Null>	<Null>	0	66	81, 102	114.546577	35.827297
83	<Null>	<Null>	<Null>	<Null>	0	65	23, 33	114.331795	34.159882
82	<Null>	<Null>	<Null>	<Null>	0	64	38, 41	114.285286	33.772247
81	<Null>	<Null>	<Null>	<Null>	0	63	25, 28	114.015938	33.57962
80	<Null>	<Null>	<Null>	<Null>	0	62	61	113.751152	35.350624
79	<Null>	<Null>	<Null>	<Null>	0	61	16, 18	113.718384	35.343185
78	<Null>	<Null>	<Null>	<Null>	0	60	13, 27	113.641785	33.422657
77	<Null>	<Null>	<Null>	<Null>	0	59	29	113.592438	33.689171
76	<Null>	<Null>	<Null>	<Null>	0	58	47, 57	113.408653	35.01804
75	新野刁河堂	<Null>	<Null>	<Null>	0	55	89	112.314255	32.439262
74	伊洛汇合处	<Null>	<Null>	<Null>	0	9	101	112.792969	34.682293

```
pointArr = {ndarray} [(81, 6.0, 114.52749634, 35.60943985, u'None'),
[0:97] = {list} [(81, 6.0, 114.52749634, 35.60943985, u'None'),
00 = {void} (81, 6.0, 114.52749634, 35.60943985, u'None')
01 = {void} (82, 6.0, 114.49967194, 35.5792923, u'None')
02 = {void} (83, 1.0, 115.61068726, 34.04090118, u'None')
03 = {void} (84, 1.0, 115.25292206, 33.98281097, u'None')
04 = {void} (85, 4.0, 114.83731079, 32.84949112, u'None')
05 = {void} (86, 3.0, 114.70394135, 33.20602798, u'None')
06 = {void} (87, 3.0, 114.66423035, 35.0655899, u'None')
07 = {void} (88, 4.0, 114.60600281, 35.13345718, u'None')
08 = {void} (89, 6.0, 111.86670685, 32.70166397, u'None')
09 = {void} (90, 4.0, 116.38637543, 33.80583954, u'None')]
```

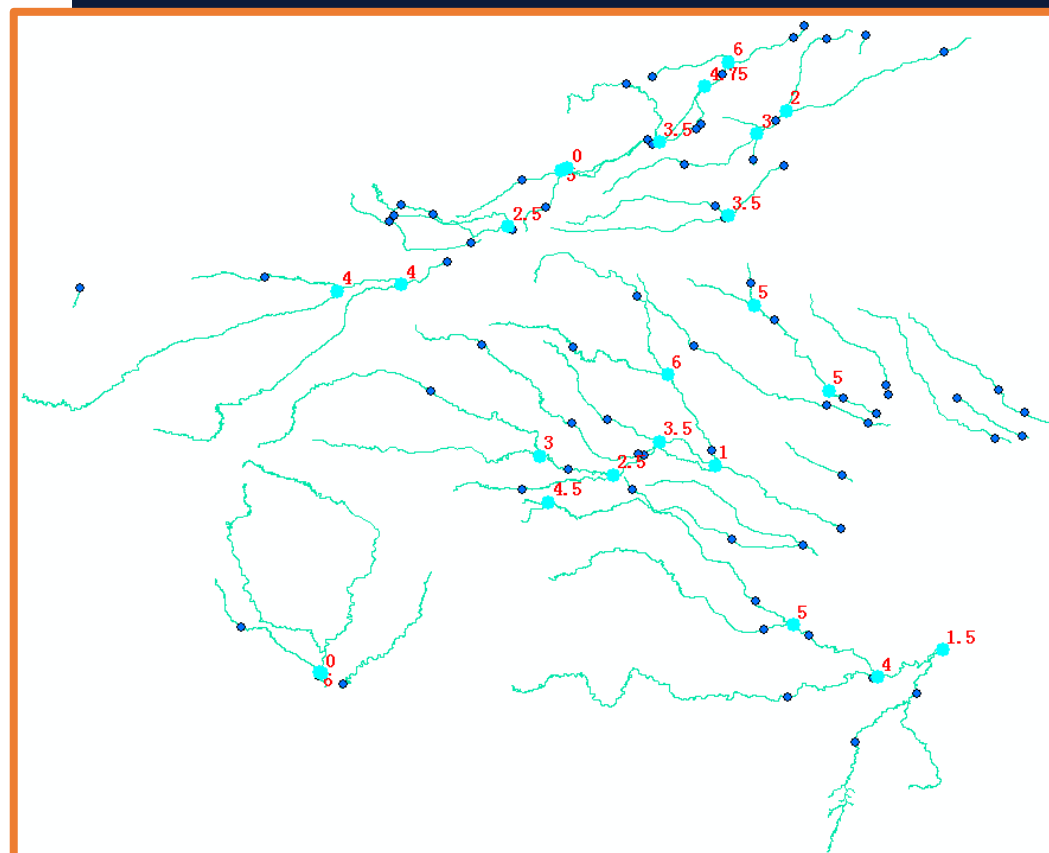
版本数据更新和NumPy应用

版本数据编辑

```
Executing: editVersionedData "D:/EnvironmentApplication/Connection to localhost.sde/mypostgresdb.sde.XYpoints_all"
Start Time: Thu Apr 19 16:12:56 2018
Running script editVersionedData...
Objects in this class cannot be updated outside an edit session [mypostgresdb.sde.XYpoints_all]
```

```
fields = ('VAL', 'MYID')
queryfilter = "PARENTID is not null"
#对于版本数据不用会话编辑会报错
'''
with arcpy.da.UpdateCursor(pointfeature, fields, queryfilter) as cursor:
    for row in cursor:
        pointid = row[1]
        for interitem in interpointarr:
            if interitem["MYID"] == pointid:
                row[0] = pointArr[pointArr["MYID"] == int(pointid)][0]["VAL"]
                break
        cursor.updateRow(row)
'''
```

```
#为编辑会话提供工作空间
workspace = os.path.dirname(pointfeature)
edit = arcpy.da.Editor(workspace)
#启动编辑会话 是否允许撤销重做/是否多用户操作
edit.startEditing(False, True)
#启动编辑操作
edit.startOperation()
with arcpy.da.UpdateCursor(pointfeature, fields, queryfilter) as cursor:
    for row in cursor:
        pointid = row[1]
        for interitem in interpointarr:
            if interitem["MYID"] == pointid:
                row[0] = pointArr[pointArr["MYID"] == int(pointid)][0]["VAL"]
                break
        cursor.updateRow(row)
#停止编辑操作
edit.stopOperation()
#停止编辑会话
edit.stopEditing(True)
```



矢量数据检查入库

- 基础检查

- 要素几何类型
- 空间参考系统
- 数据空间范围
-

- 要素类检查

- 字段类型
- 字段值域
- 关系类
-

- 拓扑检查（点、线、面）

- 自相交
- 伪结点、悬挂点
- 压盖相交
- 包含
-

数据编辑处理

Polygon

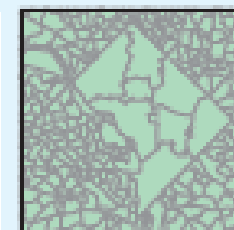
Must not overlap

Polygons must not overlap within a feature class or subtype. Polygons can be disconnected or touch at a point or touch along an edge.



Polygon errors are created from areas where polygons overlap.

Use this rule to make sure that no polygon overlaps another polygon in the same feature class or subtype.

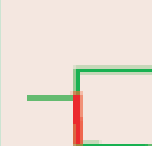
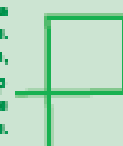


A voting district map cannot have any overlaps in its coverage.

Line

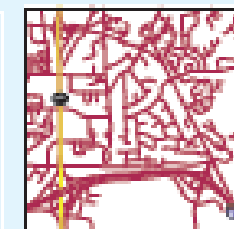
Must not self-overlap

Lines must not overlap themselves within a feature class or subtype. Lines can touch, intersect, and overlap lines in another feature class or subtype.



Line errors are created where lines overlap themselves.

Use this rule with lines whose segments should never occupy the same space as another segment on the same line.

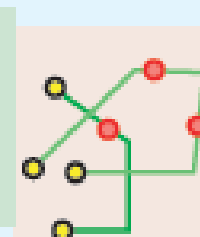
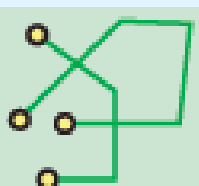


For transportation analysis, street and highway segments of the same feature should not overlap themselves.

Point

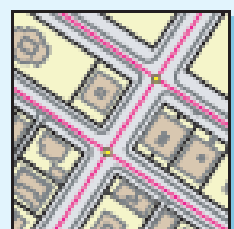
Must be covered by endpoint of

Points in one feature class or subtype must be covered by the ends of lines in another feature class or subtype.



Point errors are created on the points that are not covered by the ends of lines.

Use this rule when you want to model points that are coincident with the ends of lines.



Street intersections must be covered by the endpoints of street centerlines.

矢量数据检查入库

基础检查

- 属性描述 (**arcpy.Describe**)

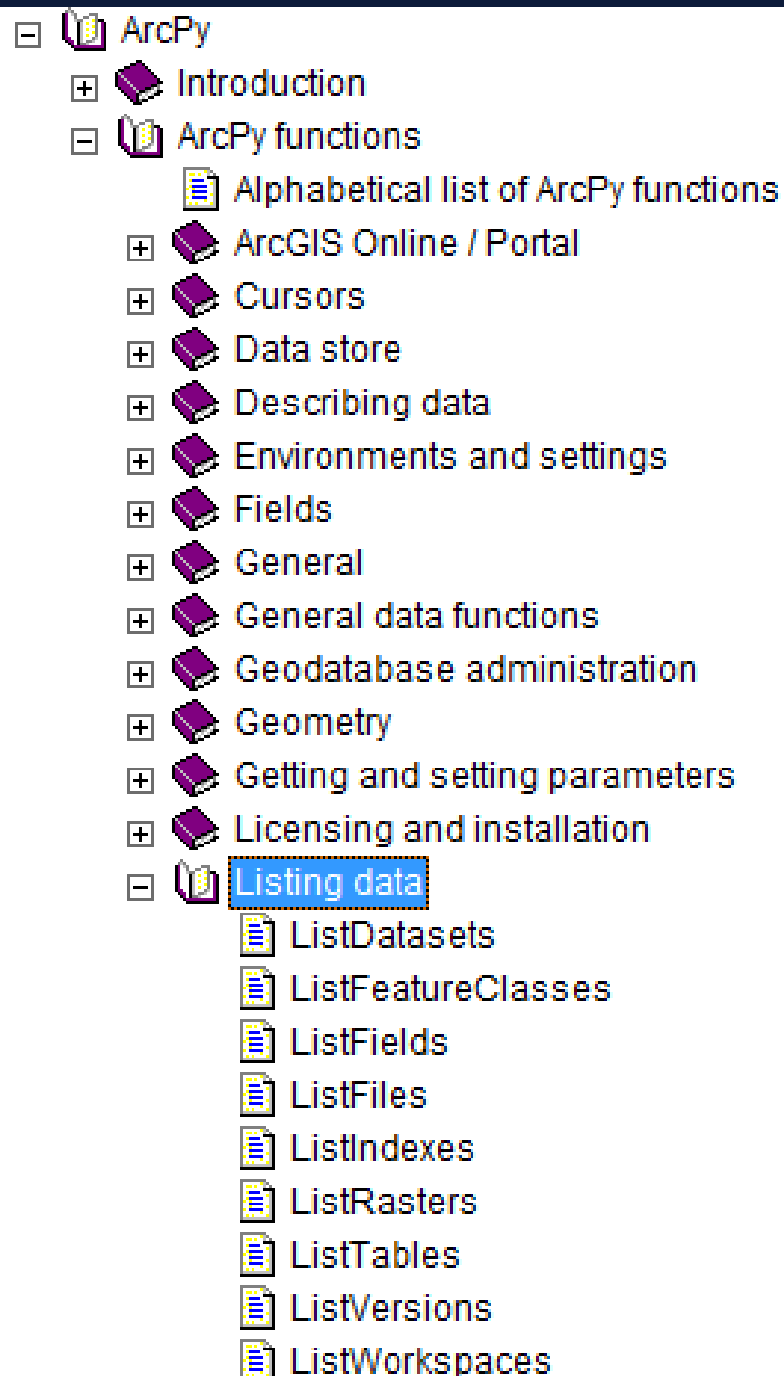
FeatureClass/GDB FeatureClass/Table/Dataset

- featureType
- hasSpatialIndex
- shapeFieldName
- shapeType
- spatialReference
-

```
def checkDataTypeAndSpatialRef(logger, landUseFeatureClass):  
    desc = arcpy.Describe(landUseFeatureClass)  
    #####第一步: 要素类型检查#####  
    #要素的几何类型  
    shapeType = desc.shapeType  
    #要素类型  
    featureType = desc.featureType  
    if(featureType=="Simple" and shapeType=="Polygon"):  
        arcpy.AddMessage("{0}数据类型检查, 符合简单面状要素要求!".format(landUseFeatureClass))  
        logger.info("{0}数据类型检查, 符合简单面状要素要求!".format(landUseFeatureClass))  
    else:  
        arcpy.AddMessage("{0}数据类型检查, 不符合简单面状要素要求!".format(landUseFeatureClass))  
        logger.error("{0}数据类型检查, 不符合简单面状要素要求!".format(landUseFeatureClass))  
  
    #####第二步: 要素空间参考#####  
    #projectionName投影坐标系名称 GCSName地理坐标系名称  
    spatial_ref = desc.spatialReference  
    if spatial_ref.name == "Unknown":  
        arcpy.AddMessage("{0}具有未知的空间参考".format(landUseFeatureClass))  
        logger.error("{0}具有未知的空间参考".format(landUseFeatureClass))  
    elif(spatial_ref.name == "GCS_China_Geodetic_Coordinate_System_2000"):  
        arcpy.AddMessage("{0} : {1}符合空间参考要求".format(landUseFeatureClass, spatial_ref.name))  
        logger.info("{0} : {1}符合空间参考要求".format(landUseFeatureClass, spatial_ref.name))  
    else:  
        arcpy.AddMessage("{0} : {1}不符合空间参考要求".format(landUseFeatureClass, spatial_ref.name))  
        logger.error("{0} : {1}不符合空间参考要求".format(landUseFeatureClass, spatial_ref.name))
```

矢量数据检查入库

要素属性结构检查



```
import arcpy

feature_class = "c:/data/counties.shp"

# Create a list of fields using the ListFields function
fields = arcpy.ListFields(feature_class)

# Iterate through the list of fields
for field in fields:
    # Print field properties
    print("Field:      {0}".format(field.name))
    print("Alias:      {0}".format(field.aliasName))
    print("Type:        {0}".format(field.type))
    print("Is Editable: {0}".format(field.editable))
    print("Required:    {0}".format(field.required))
    print("Scale:       {0}".format(field.scale))
    print("Precision:   {0}".format(field.precision))
```

```
import arcpy

domains = arcpy.da.ListDomains("C:/Boston/Boston.gdb")

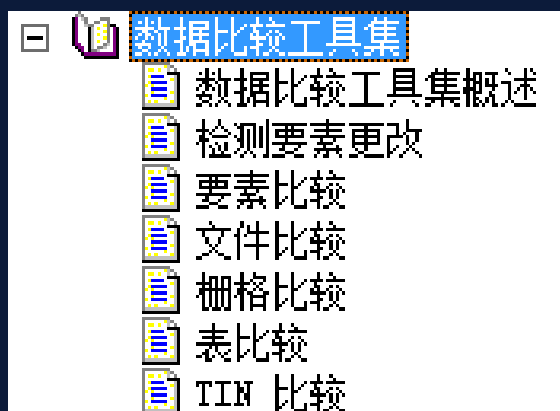
for domain in domains:
    print('Domain name: {0}'.format(domain.name))
    if domain.domainType == 'CodedValue':
        coded_values = domain.codedValues
        for val, desc in coded_values.items():
            print('{0} : {1}'.format(val, desc))
    elif domain.domainType == 'Range':
        print('Min: {0}'.format(domain.range[0]))
        print('Max: {0}'.format(domain.range[1]))
```

矢量数据检查入库

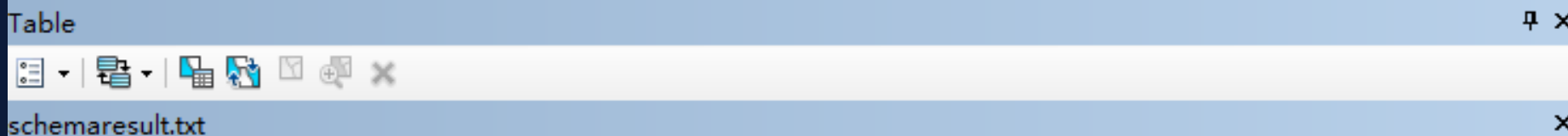
要素属性结构检查

要素比较 (**arcpy.FeatureCompare**)

- ALL — 将比较要素类的所有属性。这是默认设置
- GEOMETRY_ONLY — 仅比较要素类的几何
- ATTRIBUTES_ONLY — 仅比较属性及属性值
- SCHEMA_ONLY — 仅比较要素类的方案
- SPATIAL_REFERENCE_ONLY — 仅比较两个要素类的空间参考



```
def checkAttrSchema(targetLandUseFeatureClass, landUseFeatureClass):  
    #####第三步：要素属性结构检查#####  
    #得到总体描述  
    out_compare_file="D:/developdemo/dev2018/data/schemaresult.txt"  
    compare_result = arcpy.FeatureCompare_management(targetLandUseFeatureClass, landUseFeatureClass, "OBJECTID",  
    #result.getOutput(1) 获取此工具的状态未检测到差异时将显示 'true' 值；检测到差异时将显示 'false' 值  
    arcpy.AddMessage(compare_result.getOutput(1))  
    arcpy.AddMessage(arcpy.GetMessages())
```



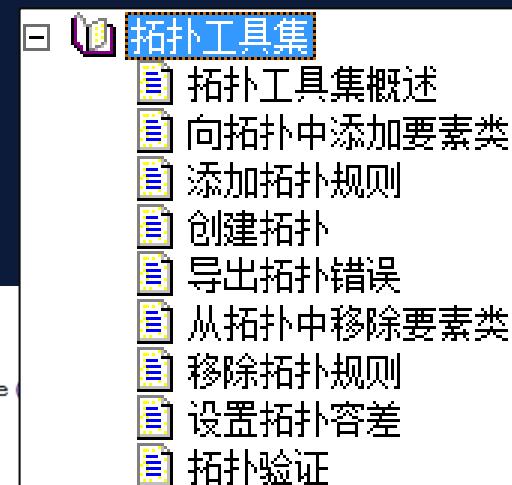
Has_error	Identifier	Message
true	Table	Tables have different number of fields
false	GeometryDef	GeometryDefs are the same
false	Field	Field properties are the same

建设用地变更

Has_error	Identifier	Message
true	FeatureClass	Feature class extents are different
false	FeatureClass	Shape types are the same
false	FeatureClass	Feature types are the same
false	Table	Table row counts are the same

矢量数据检查入库

拓扑检查



1、创建数据集

2、导入需要拓扑检查数据

3、创建拓扑

4、向拓扑中添加要素类

5、添加拓扑规则

6、验证拓扑

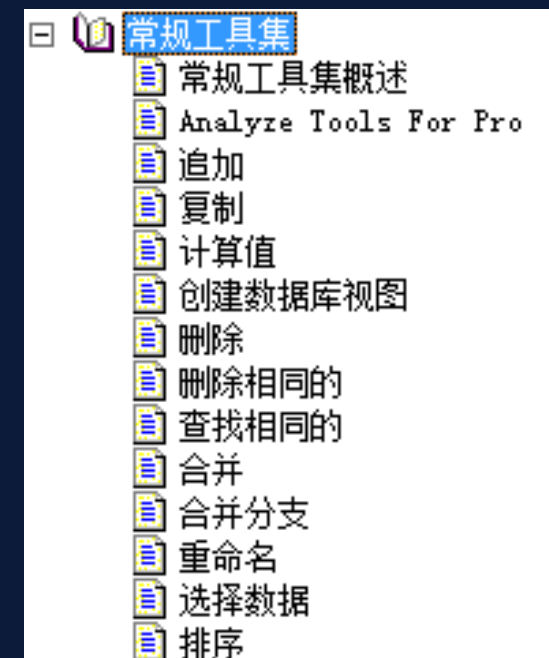
7、导出拓扑错误

```
def checkDataTopology(landUseFeatureClass, buildingFeatureClass, topoGdbPath):  
    #####第四步, 拓扑检查#####  
    #创建拓扑检查的数据集 "GCS_China_Geodetic_Coordinate_System_2000"  
    nowTimeStr=datetime.datetime.now().strftime('%Y-%m-%d %H:%M:%S').replace("-", "").replace(":", "")  
    outSpRef = arcpy.SpatialReference(4490)  
    featureDatasetName="landUse"+nowTimeStr  
    topologyFeatureDataset = topoGdbPath + "/" + featureDatasetName  
    topologyName = "landuse_Topology"+nowTimeStr  
    arcpy.CreateFeatureDataset_management(topoGdbPath, featureDatasetName, outSpRef)  
    #拓扑路径  
    in_topology = topologyFeatureDataset + "/" + topologyName  
    #原始待检查土地利用数据  
    #获取文件名称 desc.name=desc.file=desc.baseName desc.extension desc.path  
    landUseDesc = arcpy.Describe(landUseFeatureClass)  
    landUseFileNameNoExt = landUseDesc.baseName  
    buildingDesc = arcpy.Describe(buildingFeatureClass)  
    buildingFileNameNoExt = buildingDesc.baseName  
    #数据集中需要导入进行拓扑检查的数据  
    landUseTopoFeature = topologyFeatureDataset + "/" + landUseFileNameNoExt  
    #根据土地利用类型抽取水田  
    expression = "CLASS='水田'"  
    out_Layer = "selectlayer"  
    arcpy.MakeFeatureLayer_management(landUseFeatureClass, out_Layer, expression, "", "")  
    if int(arcpy.GetCount_management(out_Layer).getOutput(0)) > 0:  
        arcpy.CopyFeatures_management(out_Layer, landUseTopoFeature)  
    buildingTopoFeature = topologyFeatureDataset + "/" + buildingFileNameNoExt  
    arcpy.CopyFeatures_management(buildingFeatureClass, buildingTopoFeature)  
    #创建拓扑规则  
    arcpy.CreateTopology_management(topologyFeatureDataset, topologyName, "")  
    #添加拓扑规则适用的空间数据  
    arcpy.AddFeatureClassToTopology_management(in_topology, landUseTopoFeature, 1, 1)  
    arcpy.AddFeatureClassToTopology_management(in_topology, buildingTopoFeature, 1, 1)  
    #针对土地利用数据进行单项检查  
    #arcpy.AddRuleToTopology_management(in_topology, "Must Not Have Gaps (Area)", landUseTopoFeature, "", "#", "")  
    arcpy.AddRuleToTopology_management(in_topology, "Must Not Overlap (Area)", landUseTopoFeature, "", "#", "")  
    #针对建设用地数据进行压盖检查  
    arcpy.AddRuleToTopology_management(in_topology, "Must Not Overlap With (Area-Area)", landUseTopoFeature, "", buildingTopoFeature, "")  
    #针对企业级地理数据库, 拓扑分析的要素集不能注册版本  
    arcpy.ValidateTopology_management(in_topology, "Full_Extent")  
    #将有拓扑错误的数据导出  
    arcpy.ExportTopologyErrors_management(in_topology, topologyFeatureDataset, "TopologyError")
```

矢量数据检查入库

数据入库

- 追加记录 (**arcpy.Append**)
- 更新属性信息 (**arcpy.da.UpdateCursor**)



```
def data2TargetGDB(targetLandUseFeatureClass, landUseFeatureClass):  
    #####第五步: 入库更新#####  
    arcpy.Append_management(landUseFeatureClass, targetLandUseFeatureClass, "NO_TEST", '', "")  
    queryfilter = "timeVersion='"  
    fields = ['timeVersion']  
    with arcpy.da.UpdateCursor(targetLandUseFeatureClass, fields, queryfilter) as cursor:  
        for row in cursor:  
            row[0] = datetime.datetime.now().strftime('%Y-%m-%d %H:%M:%S')  
            cursor.updateRow(row)
```

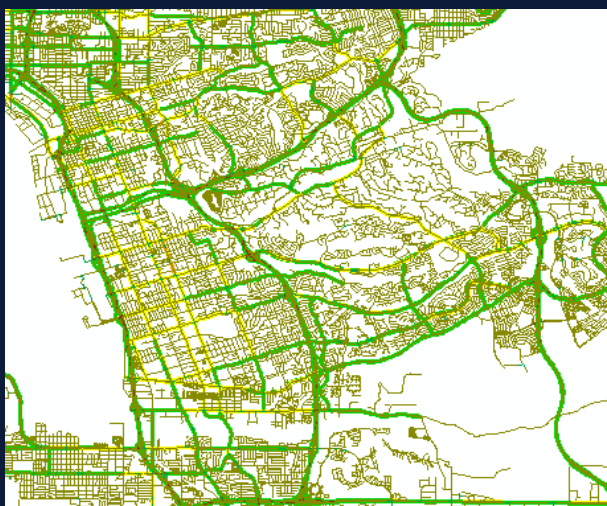
地理分析处理

地理分析处理

地理分析应用

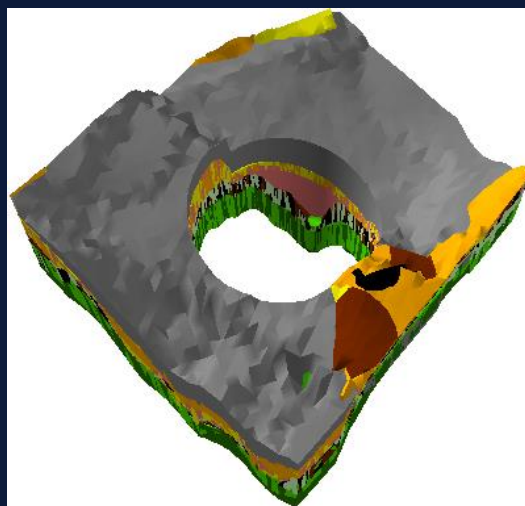
- 常见空间分析

- 适宜性分析
- 相关性分析
- 密度分析
- 邻域分析
- 水文分析
- 网络分析



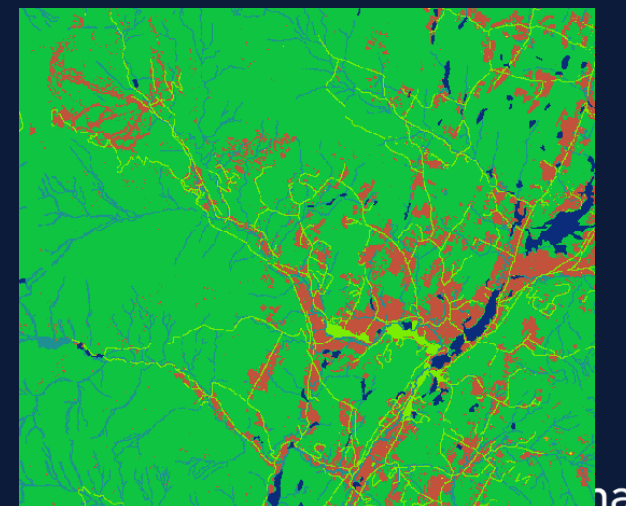
- 3D分析

- 地形分析
- 3D建模
- 可见性分析
- 填挖方分析



- 栅格处理分析

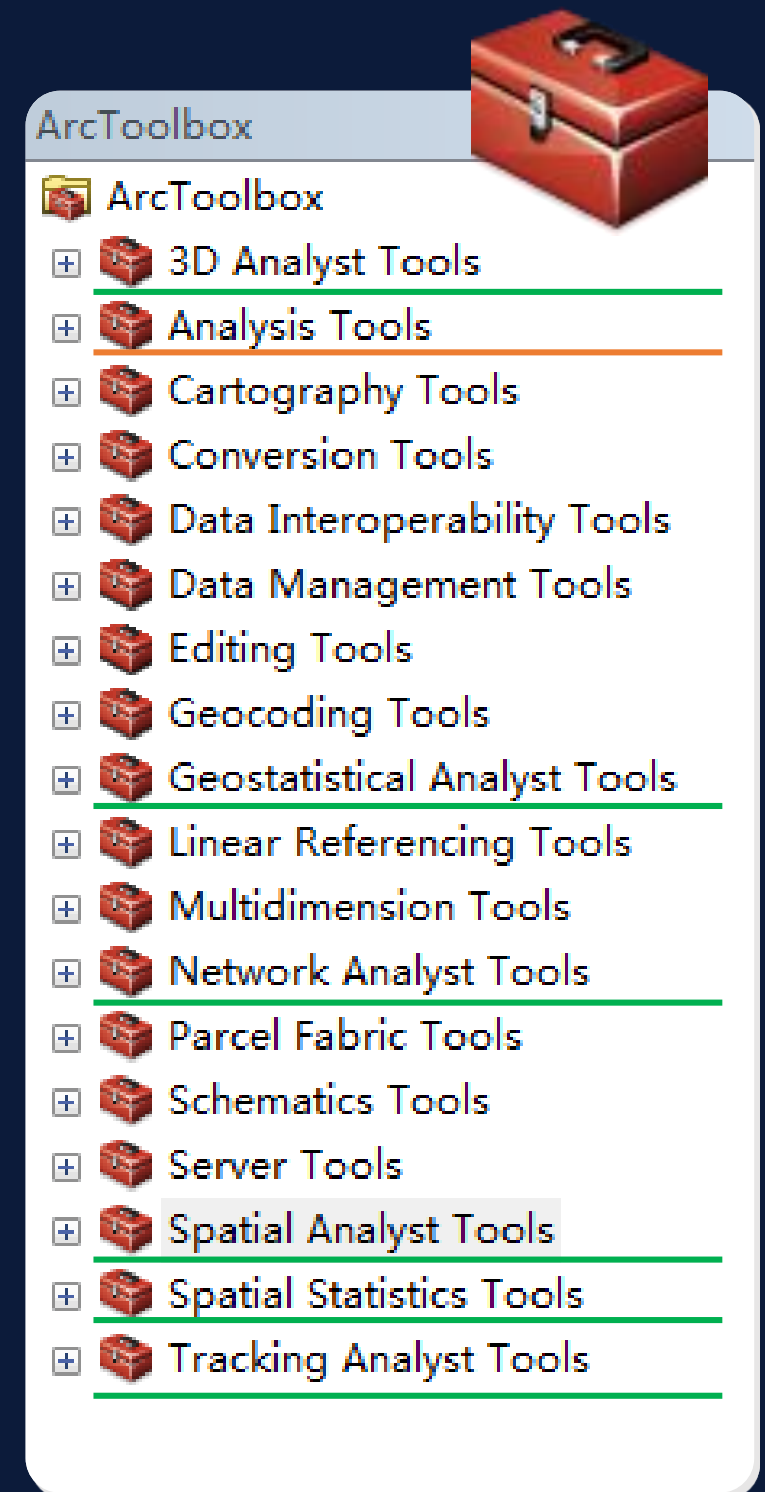
- 插值处理
- 影像分类
- 统计分析



地理分析处理

地理分析工具

- 主模块
 - 常用分析工具
- 扩展模块
 - 空间分析
 - 空间统计分析
 - 地统计分析
 - 3D分析
 - 网络分析
 - 追踪分析



空间分析注意事项

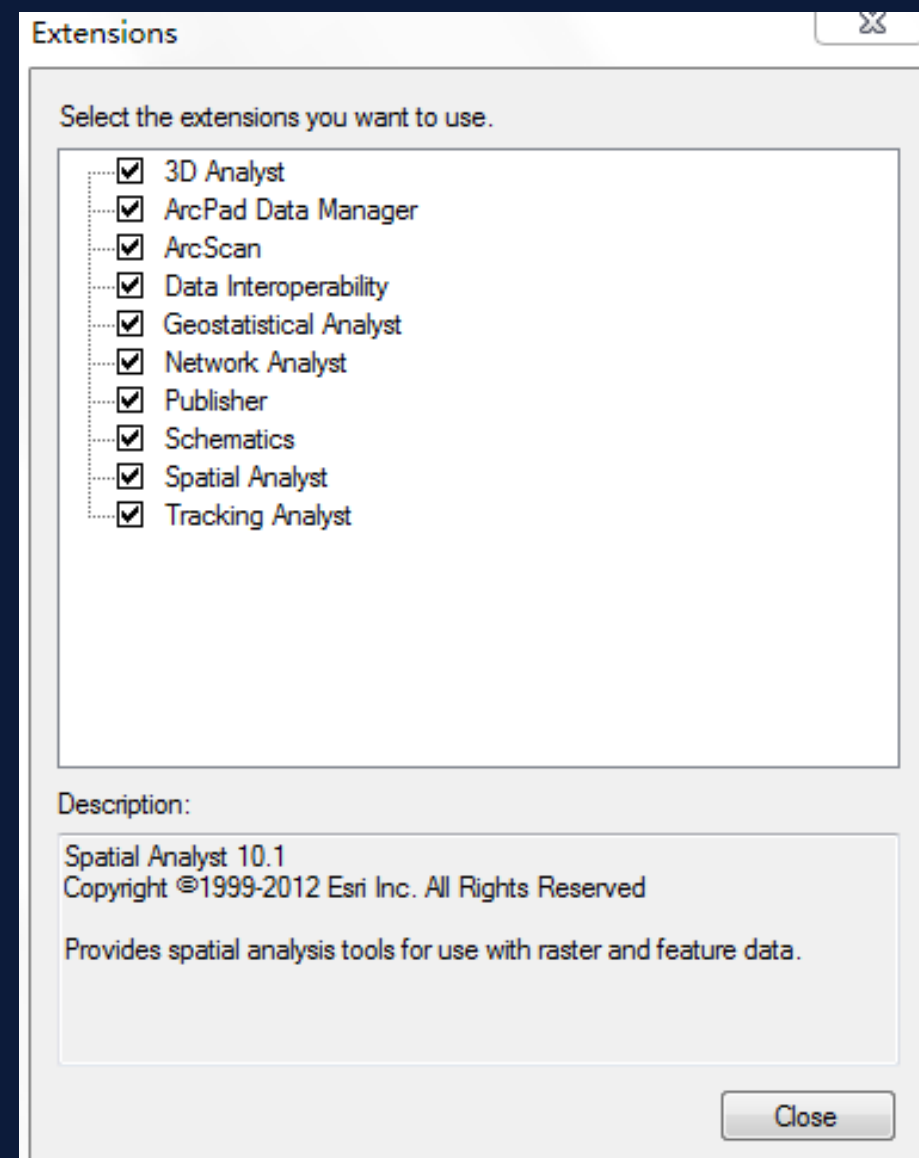
(一) 扩展模块授权

扩展模块：`arcpy.CheckOutExtension("Spatial")`

Extension	Extension Code
ArcGIS 3D Analyst	3D
ArcGIS Data Interoperability 10.5.1 for Desktop	DataInteroperability
ArcGIS Data Reviewer for Desktop	Datareviewer
ArcGIS for Aviation: Airports	Airports
ArcGIS for Aviation: Charting	Aeronautical
ArcGIS for Maritime: Bathymetry	Bathymetry
ArcGIS for Maritime: Charting	Nautical
ArcGIS Geostatistical Analyst	GeoStats
ArcGIS Network Analyst	Network
ArcGIS Spatial Analyst	Spatial
ArcGIS Schematics	Schematics
ArcGIS Tracking Analyst	Tracking

Data Type	Explanation
String	There are four possible returned values for CheckExtension: - Available —The requested license is available to be set. - Unavailable —The requested license is unavailable to be set. - NotLicensed —The requested license is not valid. - Failed —A system failure occurred during request.

```
#验证扩展模块权限
if arcpy.CheckExtension("Spatial") == "Available":
    #获取授权
    arcpy.CheckOutExtension("Spatial")
    arcpy.AddMessage("Spatial Analyst license is available")
else:
    arcpy.AddMessage("Spatial Analyst license is unavailable")
```



空间分析注意事项

(二) 环境变量设置

环境变量：`arcpy.env`

```
arcpy.env.workspace = "c:/avalon/data"
```

属性	说明
cellSize (读写)	支持“像元大小”环境设置的工具可以设置在操作时使用的输出栅格像元大小或分辨率。默认输出分辨率由最粗糙的输入栅格数据集决定。
extent (读写)	支持“输出范围”环境的工具只会处理落入此设置中所指定范围内的要素或栅格。
mask (读写)	支持“掩膜”环境的工具只会考虑运行过程中落入分析掩膜范围内的像元。
scratchFolder (只读)	临时文件夹是一个文件夹位置，可用来写入基于文件的数据（如 shapefile、文本文件和图层文件）。它是一个由 ArcGIS 管理的只读环境。
scratchGDB (只读)	临时 GDB 是可以用来写入临时数据的文件地理数据库的位置。
workspace (读写)	支持“当前工作空间”环境设置的工具将指定的工作空间用作地理处理工具输入和输出的默认位置。
overwriteOutput (读写)	管理工具在运行时是否自动覆盖任何现有输出。设置为 True 时，工具将执行并覆盖输出数据集。设置为 False 时，将不会覆盖现有输出，工具将返回错误。

空间分析注意事项

(三) 关于空间参考

空间参考：`arcpy.SpatialReference`

- 坐标系

- XY分辨率

- XY容差

- 在Analysis和Spatial Analysis工具箱，包含多个不同坐标系数据输入的工具，如`Intersect`工具，工具以第一个要素类坐标系为准转换其他输入要素类坐标
 - 设置环境变量：`output coordinate system`
 - 工具的第一个输入要素类让投影转换数量最小
- 避免未知坐标系

河流提取&几何网络追踪

- 提取矢量河流
- 创建几何网络
- 几何网络追踪
 - 追踪几何网络
 - 追踪结果输出

人工辅助

1、创建数据集

2、准备汇点数据

3、导入需要创建网络数据

4、创建几何网络

5、设置几何网络流向

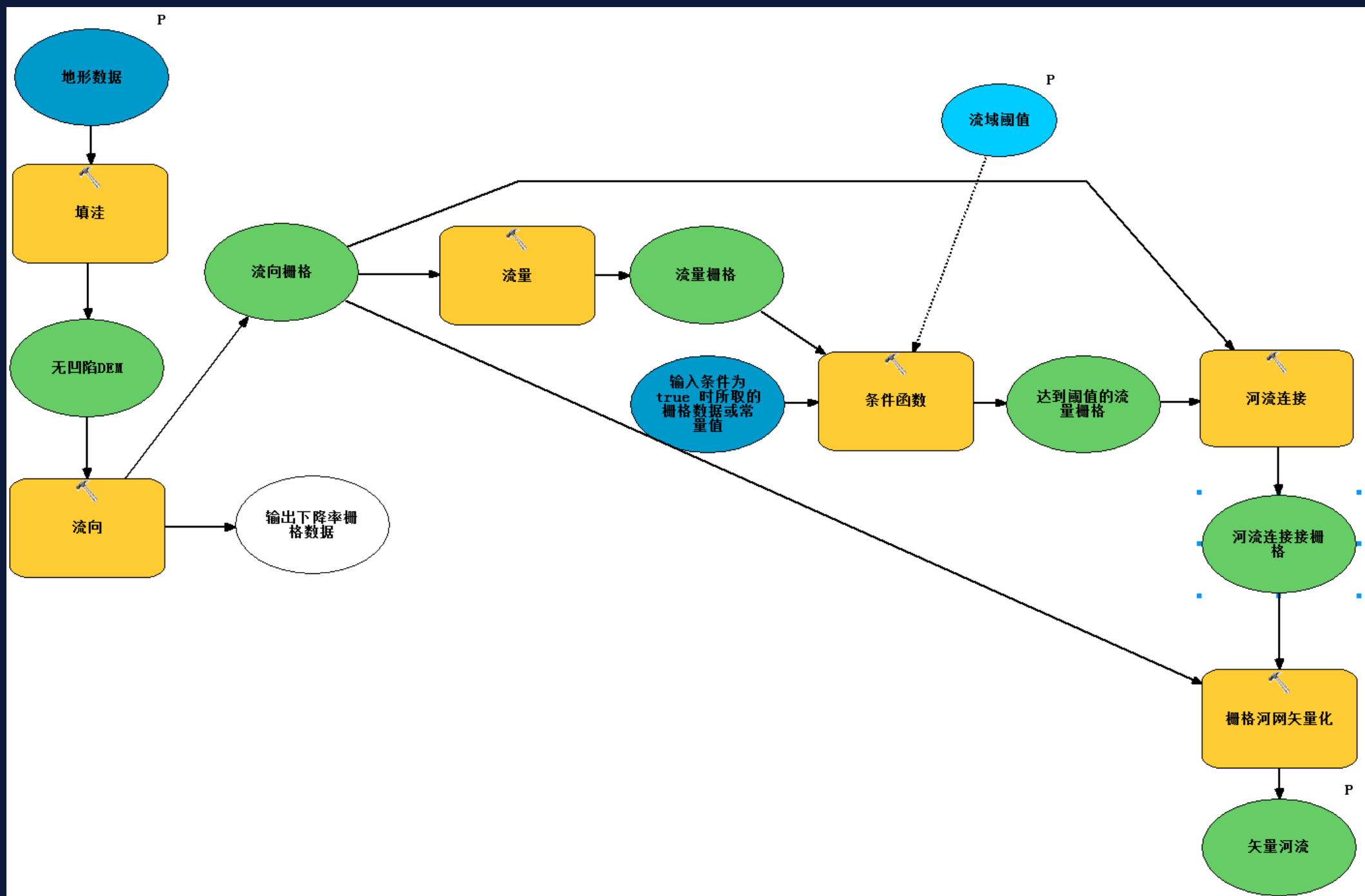
拓扑
检查

河流提取&几何网络追踪

河流提取

输入：DEM数据

输出：矢量线状河流



- 条件分析工具集
- 密度分析工具集
- 距离工具集
- 提取分析工具集
- 栅格综合工具集
- 地下水分析工具集
- 水文分析工具集**
 - 水文分析工具概述
 - 流域分析
 - 填洼
 - 流量
 - 流向
 - 水流长度
 - 汇
 - 捕捉倾泻点
 - 河流链接
 - 河网分级
 - 栅格河网矢量化
 - 分水岭
- 水文分析工具集概念
- 插值工具集
- 局部工具集
- 地图代数工具集
- 数学工具集
- 多元分析工具集
- 邻域分析工具集
- 叠加分析工具集
- 栅格创建工具集
- 重分类工具集
- 分段和分类工具集
- 太阳辐射工具集
- 表面分析工具集
- 区域分析工具集

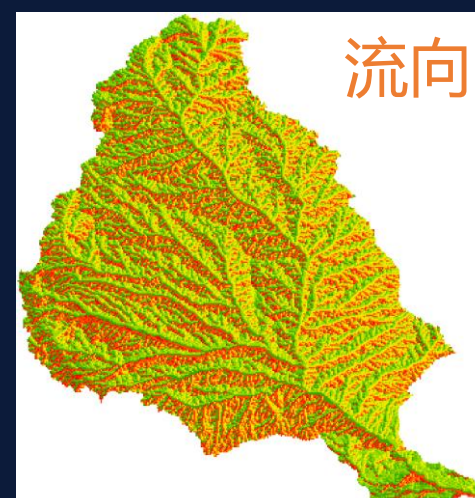
河流提取&几何网络追踪

河流提取

输入：DEM数据

输出：矢量线状河流

```
def extractRiverFeature(demData, thresholdValue, resultRiver):  
    # 填洼栅格:  
    filledDEM = tempDataPath + "/filledDEM"  
    # 流向栅格  
    flowDirectionRaster = tempDataPath + "/flowDirectionRaster"  
    # 流量栅格  
    flowAccumulationRaster = tempDataPath + "/flowAccumulationRaster"  
    # 输入条件为_true_时所取的栅格数据或常量值  
    trueRaster = 1  
    # 达到阈值的流量栅格  
    trueFlowAccumulationRaster = tempDataPath + "/trueFlowAccumulationRaster"  
    # 河流连接栅格  
    streamLinkRaster = tempDataPath + "/streamLinkRaster"  
  
    # Process: 填洼  
    outFill = Fill(demData, "")  
    outFill.save(filledDEM)  
  
    # Process: 流向  
    outFlowDirection = FlowDirection(filledDEM, "NORMAL", "")  
    outFlowDirection.save(flowDirectionRaster)  
  
    # Process: 流量  
    outFlowAccumulation = FlowAccumulation(flowDirectionRaster, "", "FLOAT")  
    outFlowAccumulation.save(flowAccumulationRaster)  
  
    # Process: 条件函数  
    outCon = Con(flowAccumulationRaster, trueRaster, "", "Value>"+thresholdValue)  
    outCon.save(trueFlowAccumulationRaster)  
  
    # Process: 河流连接  
    outStreamLink = StreamLink(trueFlowAccumulationRaster, flowDirectionRaster)  
    outStreamLink.save(streamLinkRaster)  
  
    # Process: 栅格河网矢量化  
    StreamToFeature(streamLinkRaster, flowDirectionRaster, resultRiver, "SIMPLIFY")
```



河流提取&几何网络追踪

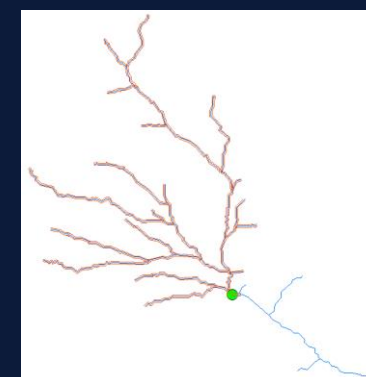
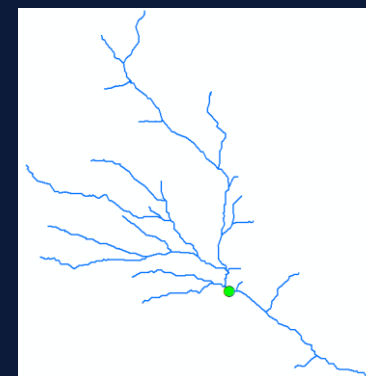
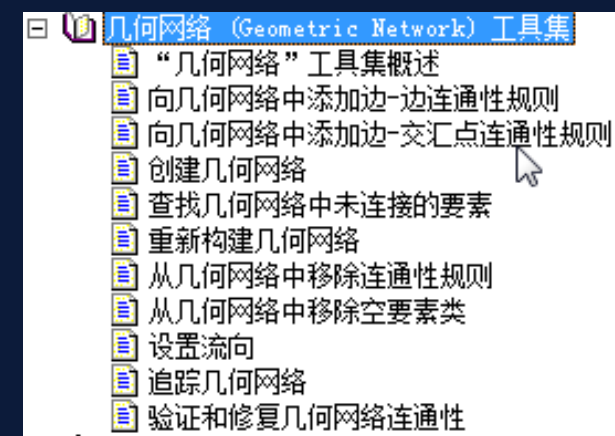
几何网络追踪

输入：几何网络&追踪起点&追踪任务

输出：追踪矢量结果

- 追踪几何网络 (**arcpy.TraceGeometricNetwork**)

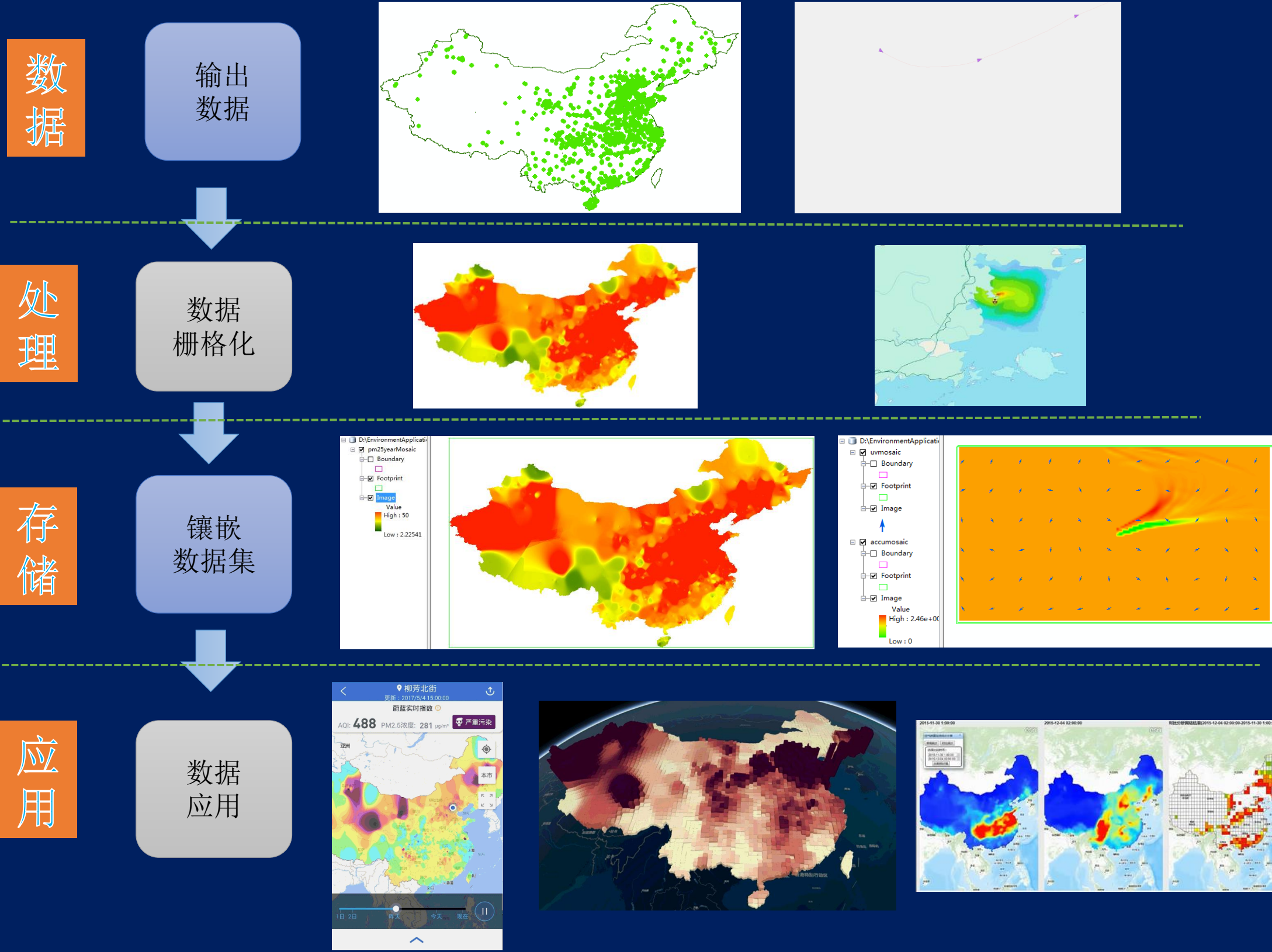
工具输出：group layer 对应几何网络中每个要素类



```
def riverTrace(in_geometric_network,in_flags,in_trace_task_type,traceResultRiver):  
    #网络追踪结果  
    riverFeatureSet_Trace = "riverFeatureSet_Trace"  
    in_barriers=""  
    in_junction_weight=""  
    in_edge_along_digitized_weight=""  
    in_edge_against_digitized_weight=""  
    in_disable_from_trace=""  
    in_trace_ends="NO_TRACE_ENDS"  
    in_trace_indeterminate_flow="NO_TRACE_INDETERMINATE_FLOW"  
    in_junction_weight_filter=""  
    in_junction_weight_range=""  
    in_junction_weight_range_not="AS_IS"  
    in_edge_along_digitized_weight_filter=""  
    in_edge_against_digitized_weight_filter=""  
    in_edge_weight_range=""  
    in_edge_weight_range_not="AS_IS"  
    Snap_Environment = [{"riverdata","EDGE","10 Meter"}]  
    arcpy.Snap_edit(in_flags, Snap_Environment)  
    #几何网络追踪  
    arcpy.TraceGeometricNetwork management(in_geometric_network, riverFeatureSet_Trace, in_flags,  
    #必须通过modelbuilder的select data工具从输出的group layer中把结果数据提取出来, 否则gp不支持gp  
    riverdata = arcpy.SelectData_management(riverFeatureSet_Trace, "riverdata")  
    riverdata_Layer = "riverdata_Layer"  
    #riverdata = riverFeatureSet_Trace  
    #创建要素图层  
    arcpy.MakeFeatureLayer_management(riverdata, riverdata_Layer, "", "", "")  
    #将几何追踪的结果保存出来  
    arcpy.CopyFeatures_management(riverdata_Layer, traceResultRiver, "", "0", "0", "0")
```

非空间数据空间化分析处理

非空间数据空间化分析处理应用流程



非空间数据空间化分析处理

Excel数据的空间化处理

输入：带位置信息Excel数据

输出：点要素和插值栅格



OBJECTID	F_lat	F_lng	PM25	time_p	time_p2
4149	31.326742	104.199648	1	2016/12/1 0:00	2016-12-01 00:00:00
3966	31.895976	117.304808	6	2016/12/1 0:00	2016-12-01 00:00:00
4016	26.549734	106.465510	11	2016/12/1 0:00	2016-12-01 00:00:00
2747	34.721088	113.742805	12	2016/12/1 0:00	2016-12-01 00:00:00
2875	36.785000	114.951000	12	2016/12/1 0:00	2016-12-01 00:00:00
3537	25.041077	101.549379	14	2016/12/1 0:00	2016-12-01 00:00:00
2512	41.833333	123.641667	15	2016/12/1 0:00	2016-12-01 00:00:00
3538	25.048180	101.543550	16	2016/12/1 0:00	2016-12-01 00:00:00
3999	30.209160	117.496121	16	2016/12/1 0:00	2016-12-01 00:00:00
3094	41.617532	120.399095	16	2016/12/1 0:00	2016-12-01 00:00:00
2506	41.756111	123.534722	16	2016/12/1 0:00	2016-12-01 00:00:00
2109	41.862500	124.038333	16	2016/12/1 0:00	2016-12-01 00:00:00
2000	43.869563	125.325003	16	2016/12/1 0:00	2016-12-01 00:00:00
2801	40.090000	116.174000	17	2016/12/1 0:00	2016-12-01 00:00:00
1983	39.914000	116.184000	17	2016/12/1 0:00	2016-12-01 00:00:00
1987	39.987000	116.287000	17	2016/12/1 0:00	2016-12-01 00:00:00
1980	39.932018	116.325121	17	2016/12/1 0:00	2016-12-01 00:00:00
1986	39.878000	116.352000	17	2016/12/1 0:00	2016-12-01 00:00:00
2808	40.127000	116.655000	17	2016/12/1 0:00	2016-12-01 00:00:00
3177	27.802967	102.346170	18	2016/12/1 0:00	2016-12-01 00:00:00
3309	22.815993	112.952383	18	2016/12/1 0:00	2016-12-01 00:00:00
1984	39.982000	116.397000	18	2016/12/1 0:00	2016-12-01 00:00:00
1985	39.937000	116.461000	18	2016/12/1 0:00	2016-12-01 00:00:00
4063	25.962186	119.516172	18	2016/12/1 0:00	2016-12-01 00:00:00
3097	41.595234	120.436163	18	2016/12/1 0:00	2016-12-01 00:00:00
3359	42.223547	123.727147	18	2016/12/1 0:00	2016-12-01 00:00:00
1992	41.371389	123.740556	18	2016/12/1 0:00	2016-12-01 00:00:00
3180	27.891171	102.236145	19	2016/12/1 0:00	2016-12-01 00:00:00
2702	24.369444	102.577778	19	2016/12/1 0:00	2016-12-01 00:00:00
2285	24.978503	102.619543	19	2016/12/1 0:00	2016-12-01 00:00:00
3075	24.712559	108.213850	19	2016/12/1 0:00	2016-12-01 00:00:00
2090	39.599288	109.781539	19	2016/12/1 0:00	2016-12-01 00:00:00
3882	23.718631	113.734804	19	2016/12/1 0:00	2016-12-01 00:00:00
2803	39.824000	116.146000	19	2016/12/1 0:00	2016-12-01 00:00:00
2820	39.899000	116.395000	19	2016/12/1 0:00	2016-12-01 00:00:00
2816	40.100000	117.120000	19	2016/12/1 0:00	2016-12-01 00:00:00
3088	42.021442	121.658560	19	2016/12/1 0:00	2016-12-01 00:00:00
2107	41.835999	123.709969	19	2016/12/1 0:00	2016-12-01 00:00:00
2108	41.882778	123.916944	19	2016/12/1 0:00	2016-12-01 00:00:00
2003	43.930372	125.317068	19	2016/12/1 0:00	2016-12-01 00:00:00
3165	26.567000	101.722670	20	2016/12/1 0:00	2016-12-01 00:00:00
3675	27.881570	102.267712	20	2016/12/1 0:00	2016-12-01 00:00:00
3178	27.838350	102.272253	20	2016/12/1 0:00	2016-12-01 00:00:00
3524	25.091530	104.880440	20	2016/12/1 0:00	2016-12-01 00:00:00

非空间数据空间化分析处理

生成点要素图层

输入：带位置信息Excel数据

输出：点要素和插值栅格

创建的图层是临时图层，如果不保存，图层在会话结束后不存在

C:\Users\Esri\Documents\ArcGIS\scratch.gdb

#中间数据输出路径

```
tempDataPath = arcpy.env.scratchGDB
timefield = "time_p2"
x_coords = "F_lng"
y_coords = "F_lat"
z_coords = ""
boundaryExtent = "73.441277 18.159829 135.086930 53.561771"
# Set the spatial reference
spRef = arcpy.SpatialReference(3857)
out_coordinate_system = arcpy.SpatialReference(4326)
```

#将excel转换成table

```
in_Table = tempDataPath+"/"+valField+"ExcelDataTable"
arcpy.ExcelToTable_conversion(in_Excel, in_Table, Sheet="Table")
```

#根据table中的x和y转成Layer,如果xy为经纬度，直接用下述方法，否则则进行投影转换

```
arcpy.MakeXYEventLayer_management(in_Table, x_coords, y_coords, out_Layer, out_coordinate_system)
excelPoint = tempDataPath + "/excelPoint"
arcpy.CopyFeatures_management(out_Layer, excelPoint)
arcpy.AddMessage("excel转换成了table")
```

#获取时间

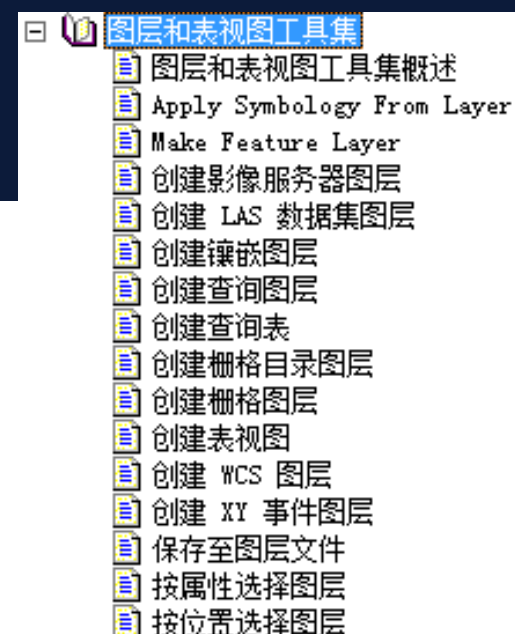
```
fcfields = [timefield]
tabledata = arcpy.da.TableToNumPyArray(in_Table, fcfields, "")
```

#获取时间字段信息

```
timelist = [x[0] for x in tabledata]
```

#获取不重复的时间

```
timelist = list(set(timelist))
```



非空间数据空间化分析处理

Idw插值处理

输入：带位置信息Excel数据

输出：点要素和插值栅格

- 栅格处理环境变量（**arcpy.env**）

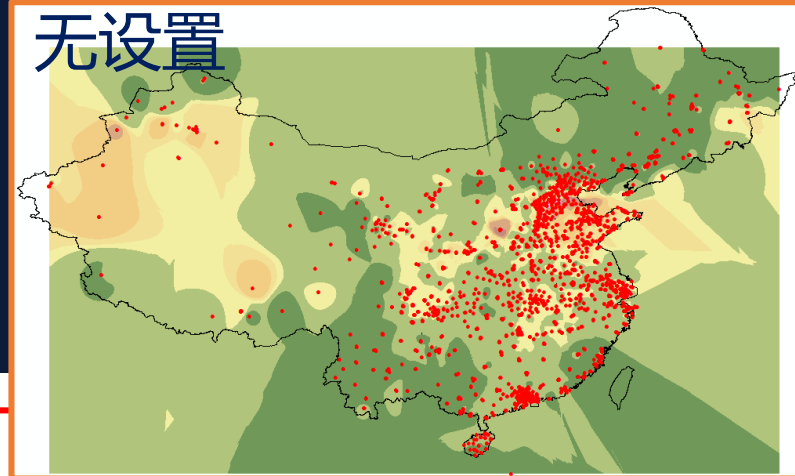
extent：控制插值处理范围

mask：限制输出栅格范围

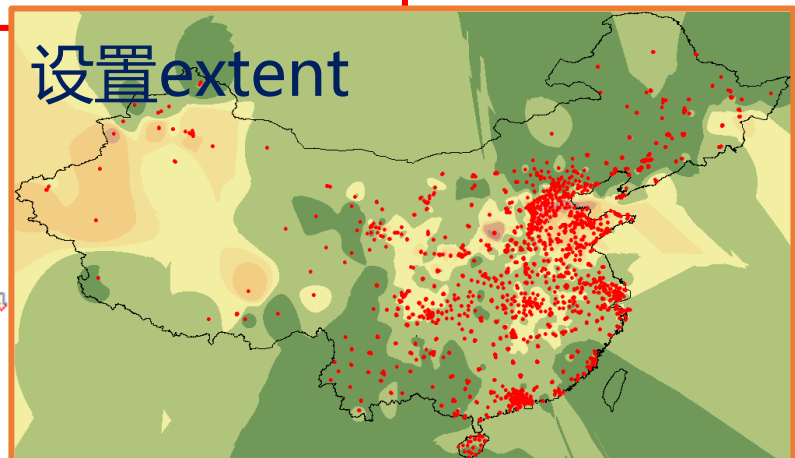
```
for i in timelist:
    expression = timefield+"=" +str(i)+"' and "+valField + "<>0"
    arcpy.SelectLayerByAttribute_management(out_Layer, "NEW_SELECTION",expression)
    if int(arcpy.GetCount_management(out_Layer).getOutput(0)) > 2:
        # Set local variables          生成栅格名称规范    201611302300    yyyymmddhhmm
        inFeatures = out_Layer
        timeStrArr = str(i).split(" ")
        timeStr = ""
        if(len(timeStrArr)>1):
            timePre = timeStrArr[0]
            timeSuf = timeStrArr[1]
            timeStr = (datetime.datetime.strptime(str(i),"%Y-%m-%d %H:%M:%S")).strftime("%Y-%m-%d %H:%M:%S")
            timeStr = timeStr.replace("/","").replace(":","").replace(" ","").replace("-","")
        else:
            timePre = timeStrArr[0]
            timeSuf = " 00:00:00"
            timeStr = (datetime.datetime.strptime((timePre+timeSuf),"%Y-%m-%d %H:%M:%S")).strftime("%Y-%m-%d %H:%M:%S")
            timeStr = timeStr.replace("/","").replace(":","").replace(" ","").replace("-","")

        outRasterName = outRasterPath+str(dem)+"_"+timeStr+".tif"
        assignmentType = "MAXIMUM"
        priorityField = ""
        #*****设置idw插值生成的栅格范围 后面需要用ResetEnv
        arcpy.env.extent = boundaryExtent
        #指定mask, 生成对应区域的栅格 栅格分析mask用来控制输出栅格范围, processing extent处理范围
        arcpy.env.mask = xzqhPolygon
        #进行插值处理
        arcpy.gp.Idw_sa(inFeatures, valField, outRasterName, ".00816878", "5", "VARIABLE 10",
        arcpy.AddMessage("生成栅格"+timeStr)
```

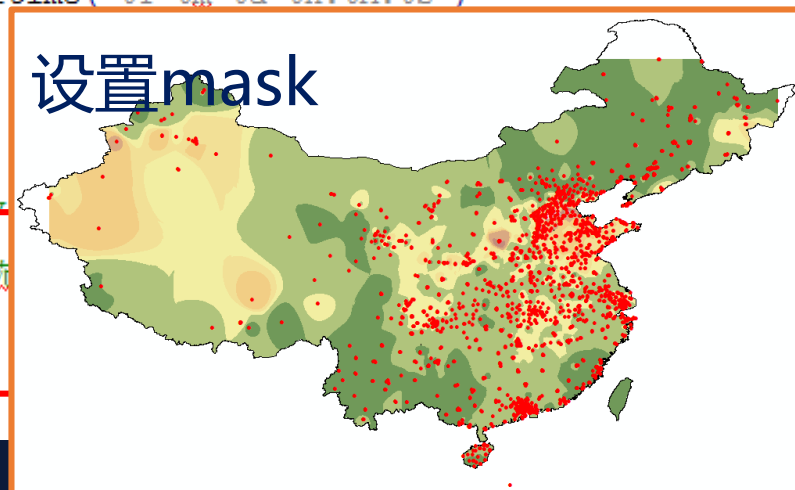
无设置



设置extent



设置mask



非空间数据空间化分析处理

栅格批量入库更新

输入：带位置信息Excel数据

输出：点要素和插值栅格

arcpy.ResetEnvironments()

```
AddRastersToMosaicDataset_management (in_mosaic_dataset, raster_type, input_path, {update_cellsize_ranges}, {update_boundary}, {update_overviews}, {maximum_pyramid_levels}, {maximum_cell_size}, {minimum_dimension}, {spatial_reference}, {filter}, {sub_folder}, {duplicate_items_action}, {build_pyramids}, {calculate_statistics}, {build_thumbnails}, {operation_description}, {force_spatial_reference}, {estimate_statistics}, {aux_inputs})
```

```
mosaicDatasetPath = "D:/developdemo/dev2018/tool/Connection to localhost.sde/mypostgresdb.sde.airMonitorData"
```

#入镶嵌数据集

```
arcpy.ResetEnvironments()  
arcpy.AddRastersToMosaicDataset_management(mosaicDatasetPath, "Raster Dataset", outRasterPath, "UPDATE_CELL_SIZES"  
arcpy.AddMessage("批量入库完成!")  
fields = ('Name', 'TIME', 'DEM', 'DVERSION', 'AIRDATATYPE')  
queryfilter = "DVERSION='-1'"
```

#批量更新记录属性

```
with arcpy.da.UpdateCursor(mosaicDatasetPath, fields, queryfilter) as cursor:  
    for row in cursor:  
        rastername = row[0]  
        arcpy.AddMessage("rastername:"+rastername)  
        timeValue = rastername.split("_")[1]  
        year = int(timeValue[0:4])  
        month = int(timeValue[4:6])  
        date = int(timeValue[6:8])  
        time = int(timeValue[8:10])  
        row[1] = datetime.datetime(year, month, date) + datetime.timedelta(hours=time)  
        row[2] = dem  
        row[3] = DversionBS  
        row[4] = valField  
        cursor.updateRow(row)
```

批量处理

非空间数据空间化分析处理

关系数据库数据的空间化处理

输入：关系数据库中数据

ArcGIS支持的数据库

1、连接数据库（直连）



2、创建XY事件图层



3、生成点要素图层



4、数据分组插值



5、栅格数据入库



6、批量更新属性

输出：点要素和插值栅格

ArcGIS不支持的数据库

1、连接数据库（Python包）



2、逐条写入数据



3、生成点要素图层



4、数据分组插值

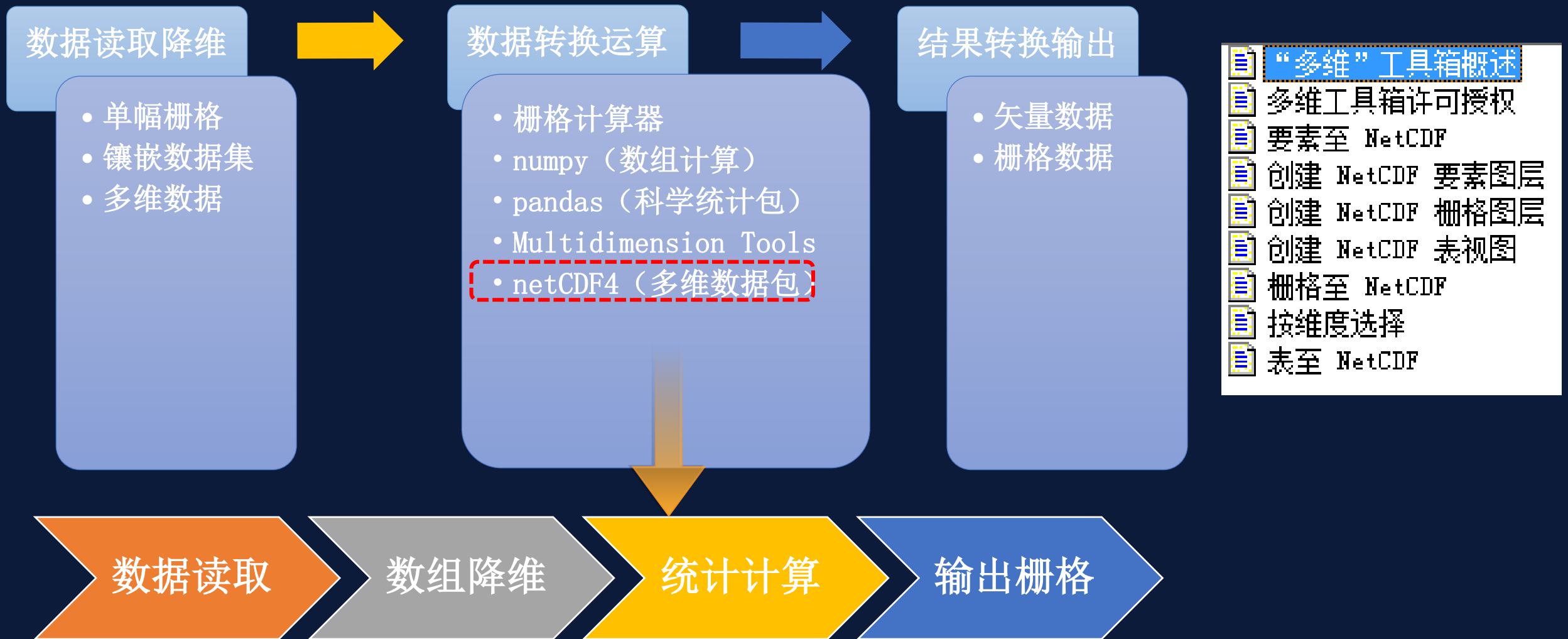


5、栅格数据入库



6、批量更新属性

栅格数据统计分析处理



栅格数据统计分析处理

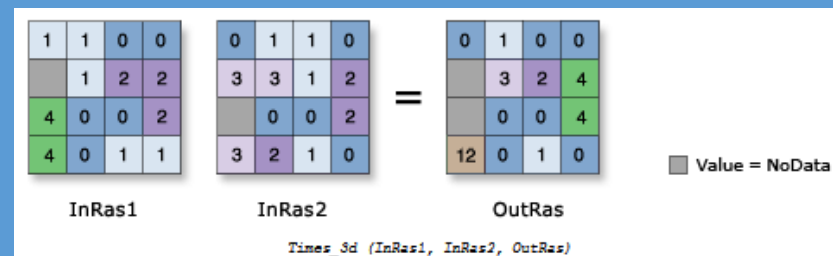
tif数据统计分析

- `arcpy.RasterToNumPyArray`, `arcpy.NumPyArrayToRaster`

```
#导出符合时间条件的栅格路径
where_clause = "(TIME >= date '"+firstDataTime+"' ) AND (TIME <= date '"+secondDataTime+"') "
export_mode = "ALL"
types_of_paths = "RASTER"
pathField = "Path"
#获取原始的tif的路径
arcpy.ExportMosaicDatasetPaths_management(sourceDatasetPath, out_table, where_clause, export_mode)
#获取符合时间条件的栅格路径列表中记录数量
tableresult = arcpy.GetCount_management(out_table)
iCount = int(tableresult.getOutput(0))
arcpy.AddMessage("导出栅格数据路径!")
#cal the meanvalue
if(iCount>0):
    cursor2 = arcpy.SearchCursor(out_table)
    iCount2 = 0
    #根据栅格个数定义二维数组
    arrWhole = numpy.ndarray([iCount,maxcols*maxrows])
    resultarr = numpy.ndarray([1,maxcols*maxrows])
    for row2 in cursor2:
        inputRasterPath2 = row2.getValue(pathField)
        inputRaster = arcpy.Raster(inputRasterPath2)
        #raster转成NumPyArray
        arr = arcpy.RasterToNumPyArray(inputRaster,nodata_to_value=0)
        #数组变换为1行N列数组
        arrWhole[iCount2] = arr.reshape(1,-1)
        iCount2 = iCount2+1
    #计算统计值
    if(calType=="mean"):
        resultarr = numpy.mean(arrWhole,axis=0)
        rastername = str(rasterpref)+"mean"
    elif(calType=="max"):
        resultarr = numpy.max(arrWhole,axis=0)
        rastername = str(rasterpref)+"max"
    elif(calType=="sum"):
        resultarr = numpy.sum(arrWhole,axis=0)
        rastername = str(rasterpref)+"sum"
    resultarr = resultarr.reshape(maxrows,maxcols)
    #结果存到scratch目录下
    meansavepath=resultFolder+"/"+rastername+".tif"
    outMeanGTE = arcpy.NumPyArrayToRaster(resultarr, firstpoint,xsize, ysize)
    outMeanGTE.save(meansavepath)
    arcpy.AddRastersToMosaicDataset_management(mosaicDatasetPath,"Raster Dataset",meansavepath,
    #更新生成的统计结果的信息
    queryfilter = "Name='"+rastername+"' AND timeversion IS NULL"
    fields = ('Name', 'statictype', 'timeversion')
    with arcpy.da.UpdateCursor(mosaicDatasetPath, fields,queryfilter) as cursor:
        for row in cursor:
            row[1] = calType
            row[2] = str(timeVersion)
            cursor.updateRow(row)
    arcpy.AddMessage("统计结果入库!")
```

NumPy

- 矩阵运算（转置、算术运算）



- 多维数组构建
- 索引
- 数组合并、分割
- 维度转换（数据降维）
- 文件加载
- 栅格转换

栅格数据统计分析处理

netCDF数据统计分析

Name	Long Name	Type
pm257.nc	pm257.nc	Local File
lat	latitude of ...	1D
lon	longitude of ...	1D
pm25	pm25	Geo2D
t	t	1D
z	z	1D

```
netcdf file:/D:/developdemo/dev2018/data/pm257.nc {
  dimensions:
    lon = 193;
    lat = 126;
    t = UNLIMITED; // (168 currently)
    z = UNLIMITED; // (12 currently)
  variables:
    double lon(lon=193);
      :long_name = "longitude of data";
      :units = "degree_east";
      :standard_name = "longitude";

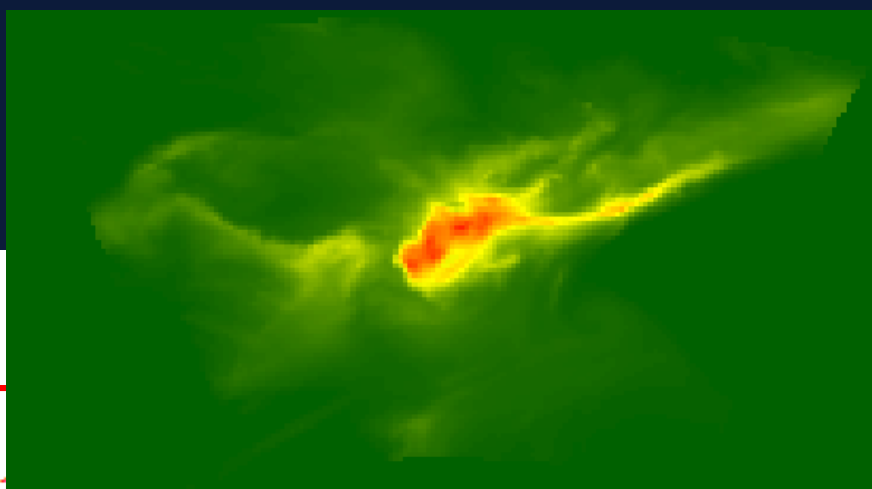
    double lat(lat=126);
      :long_name = "latitude of data";
      :units = "degree_north";
      :standard_name = "latitude";

    double t(t=168);
      :_ChunkSize = 524288; // int

    double z(z=12);
      :_ChunkSize = 524288; // int

    double pm25(t=168, z=12, lat=126, lon=193);
      :_FillValue = 0.0; // double
      :coordinates = "lon lat";
      :_ChunkSize = 1, 1, 126, 193; // int
```

```
ds = netCDF4.Dataset(ncpath, 'r', format='NETCDF4')
#获取pm25变量的值
sdata = ds.variables['pm25']
#获取各个维度变量的值
southnorthdata = ds.variables['lat'][:]
westeastdata = ds.variables['lon'][:]
timedata = ds.variables['t'][:]
bottomtopdata = ds.variables['z'][:]
#统计某一天的均值、总量、极值
indexnum = int(timeindex)+1
newdata = sdata[int(timeindex)*24:indexnum*24,:,:,]
timesize = newdata.shape[0]
topsize = newdata.shape[1]
rowsize = newdata.shape[2]
colsize = newdata.shape[3]
#进行降维操作
newdata = newdata.reshape(timesize*topsize,-1)
rasterfilename = "nc"+caltype+str(timeindex)
#统计结果转换成文本
txtfilepath = tmpFolder+"/"+rasterfilename+".txt"
if(caltype=="mean"):
    resultarr = newdata.mean(axis=0)
    resultarr = resultarr.reshape(rowsize,colsize)[:,-1,:-1]
    np.savetxt(txtfilepath,resultarr)
elif(caltype=="sum"):
    resultarr = newdata.sum(axis=0)
    resultarr = resultarr.reshape(rowsize,colsize)[:,-1,:-1]
    np.savetxt(txtfilepath,resultarr)
elif(caltype=="max"):
    resultarr = newdata.max(axis=0)
    resultarr = resultarr.reshape(rowsize,colsize)[:,-1,:-1]
    np.savetxt(txtfilepath,resultarr)
ds.close()
```



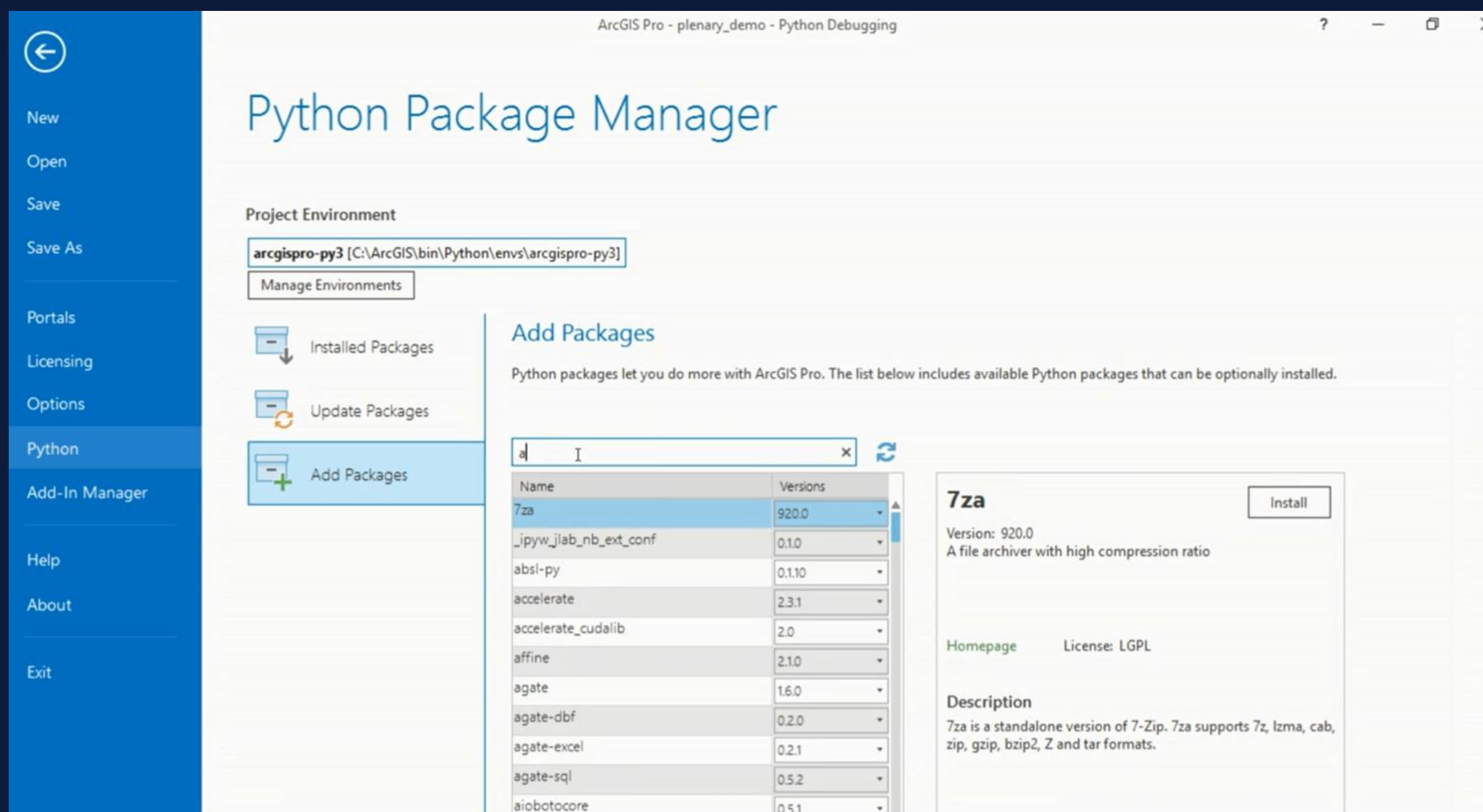
#统计结果与栅格转换

```
rasterfilename = "nc"+caltype+str(timeindex)
txtfilepath = tmpFolder+"/"+rasterfilename+".txt"
resultarr = np.loadtxt(txtfilepath)
firstpoint = arcpy.Point(46.8186456859, -16.6566636887)
myRaster = arcpy.NumPyArrayToRaster(resultarr, firstpoint, 0.6081081, 0.6081081)
savefilepath = tmpFolder+"\\"+rasterfilename+".tif"
myRaster.save(savefilepath)
```

GP工具从ArcGIS Desktop到Pro迁移

1、第三方Python包的安装

C:\Program Files\ArcGIS\Pro\bin\Python\Scripts conda install pymysql



GP工具从ArcGIS Desktop到Pro迁移

 C:\Program Files\ArcGIS\Pro\bin\Python\envs\arcgispro-py3\Scripts

2、GP工具代码从2.x到3.x的转换

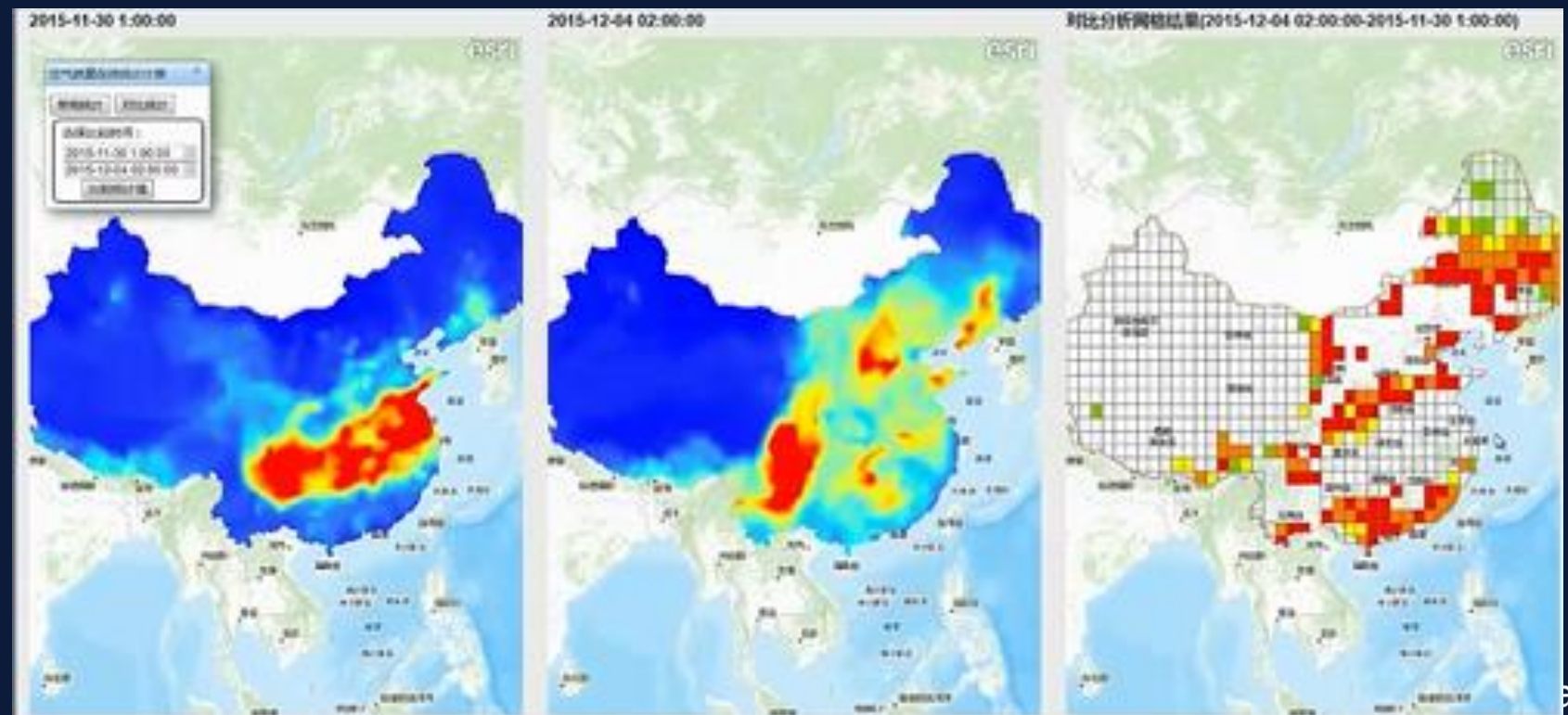
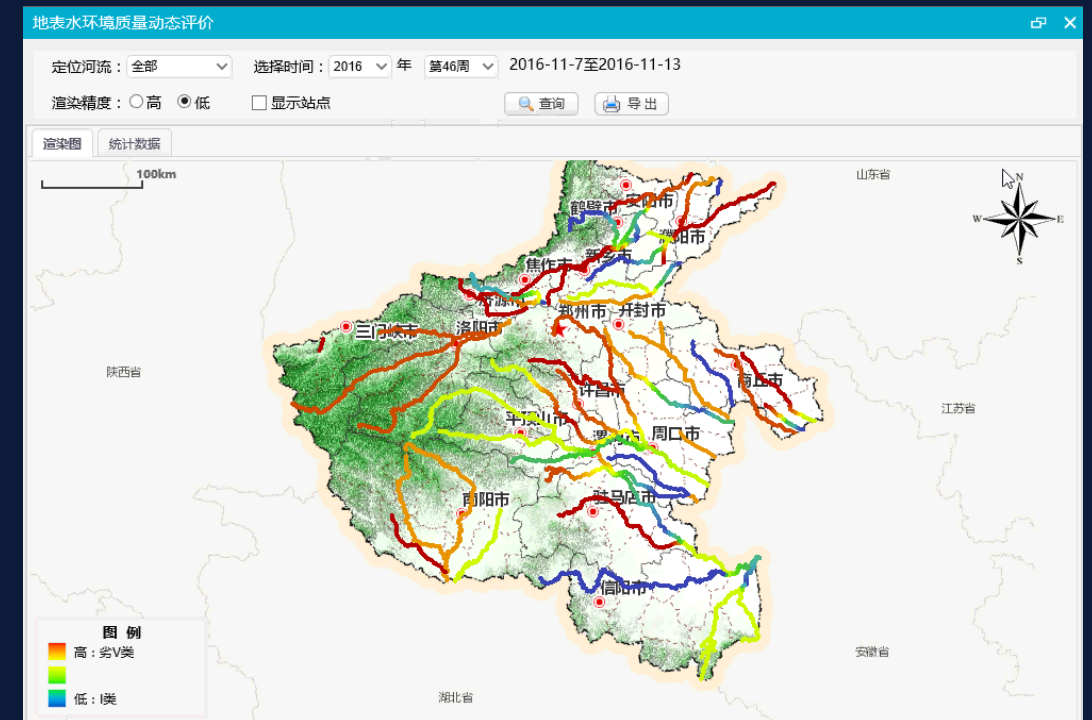
*C:\Program Files\ArcGIS\Pro\bin\Python\envs\arcgispro-py3\Scripts>2to3 -w
D:\EnvironmentApplication\qinghua\tool*

*C:\Program Files\ArcGIS\Pro\bin\Python\envs\arcgispro-py3\Scripts>2to3 -w
D:\EnvironmentApplication\qinghua\tool\exportMysqlData3x.py*

3、Pro下工具运行测试

在线GP分析处理

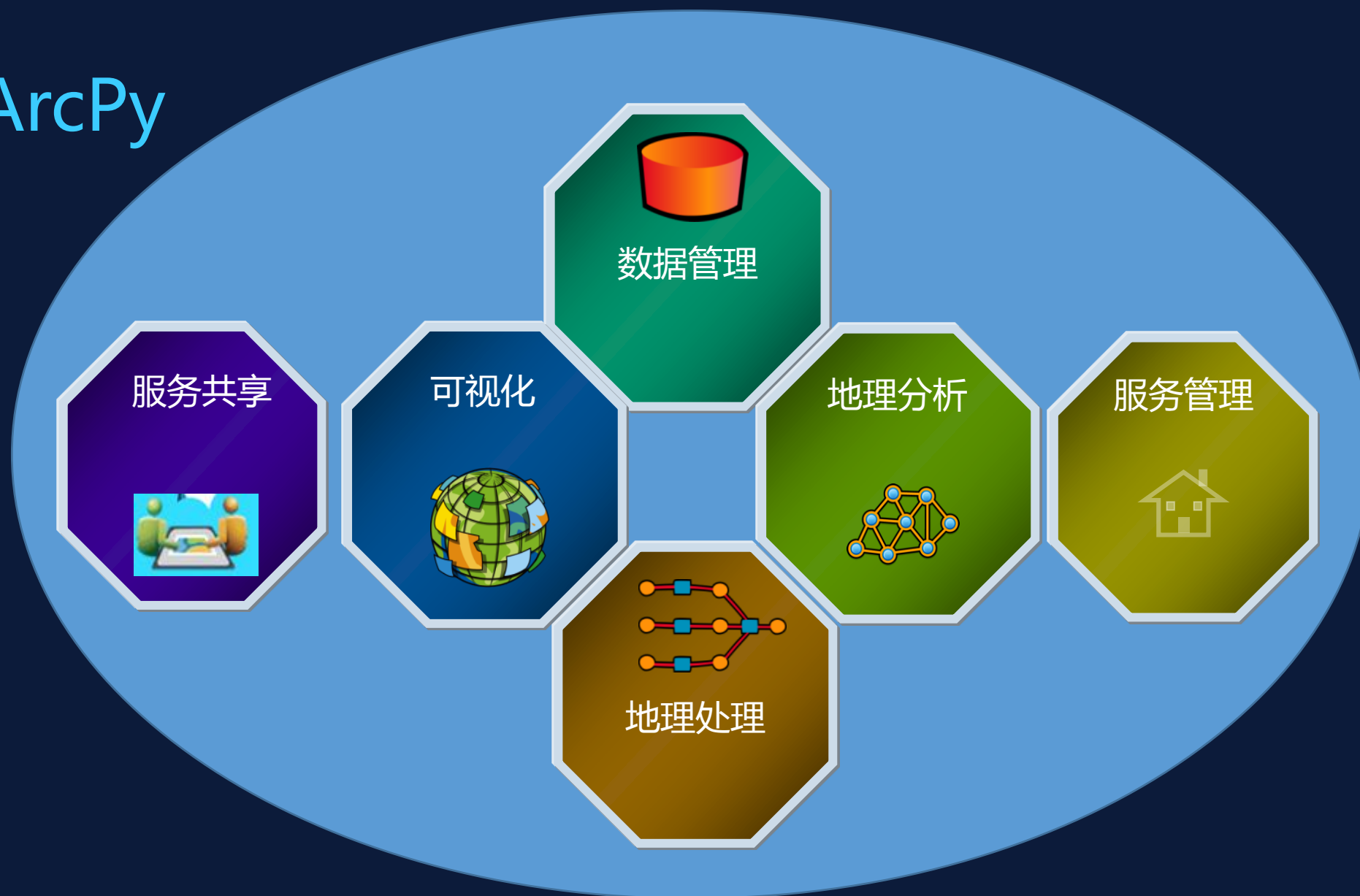
GP Service



总结

ArcGIS API for Python

ArcPy



关注

微信公众号：
环保GIS技术与应用



***THE
SCIENCE
OF
WHERE***